École Supérieure Privée d'Ingénierie et de Technologie – ESPRIT
Module : Applications Web Distribuées (MT-41)
Classe : 4DAS6 – Groupe 5 · Année universitaire 2026-2027

Mini-projet · Validation intermédiaire (semaine 4)

SkillSwap

Plateforme d'échange de compétences par crédits-temps

DOSSIER TECHNIQUE DU PROJET

Architecture microservices · Conception détaillée de MS1 à MS7

Dépôt Git : https://github.com/faryoula11/skillswap-4das6-g5

Équipe projet

Emna Trabelsi	MS1 · user-service
Amal Jemli	MS2 · skill-service
Feryel Bassem	MS3 · exchange-service
Molka Khlaifi	MS4 · credit-service
Malek Massoudi	MS5 · review-service
Wiem Jlassi	MS6 · notification-service
Toute l'équipe	MS7 · recommendation-service

Sommaire

1	**Présentation du projet**	**3**
1.1	Contexte	3
1.2	Problématique	3
1.3	Solution proposée : SkillSwap	3
1.4	Objectifs techniques	3
2	**Acteurs et fonctionnalités**	**4**
2.1	Acteurs	4
2.2	Diagramme de cas d’utilisation	4
2.3	Fonctionnalités principales	5
3	**Architecture globale**	**6**
3.1	Vue d’ensemble	6
3.2	Découpage en microservices	7
3.3	Composants d’infrastructure	7
3.4	Principes d’architecture	8
3.5	Données gérées par chaque service et choix des bases	8
4	**Technologies choisies**	**9**
5	**Sécurité**	**10**
6	**Communications entre microservices**	**11**
6.1	Communications synchrones (OpenFeign)	11
6.2	Communications asynchrones (RabbitMQ)	11
6.3	Catalogue des événements	11
6.4	Gestion des erreurs	12
7	**Conception détaillée des microservices**	**13**
7.1	MS1 · user-service	13
7.2	MS2 · skill-service	19
7.3	MS3 · exchange-service	24
7.4	MS4 · credit-service	29
7.5	MS5 · review-service	33
7.6	MS6 · notification-service	39
7.7	MS7 · recommendation-service	44
8	**Organisation du projet**	**47**
8.1	Répartition des tâches	47
8.2	Dépôt Git et gestion de projet	47
8.3	Planning prévisionnel	48
8.4	Risques et parades	48
9	**Conclusion**	**49**
Annexe	**Ports et accès de l’environnement**	**50**

1 Présentation du projet

1.1 Contexte

Beaucoup de personnes maîtrisent une compétence qu'elles pourraient transmettre et souhaitent en acquérir d'autres qu'elles n'ont pas les moyens de payer. Les cours en ligne sont payants et le partage de connaissances entre particuliers reste informel, sans suivi ni garantie de confiance.

1.2 Problématique

« Comment permettre aux utilisateurs de partager leurs connaissances et d'apprendre de nouvelles compétences sans paiement traditionnel ? »

1.3 Solution proposée : SkillSwap

SkillSwap est une plateforme d'échange de compétences par crédits-temps. Le principe est simple : **1 heure enseignée = 1 crédit gagné**, **1 heure apprise = 1 crédit dépensé**. Aucun paiement en argent n'intervient.

Le parcours nominal d'un membre se déroule en six étapes :

1. Publier ses compétences (proposées ou recherchées) ;
2. Rechercher une compétence auprès des autres membres ;
3. Demander un échange à un enseignant ;
4. L'enseignant accepte (ou refuse) la demande ; les crédits de l'apprenant sont alors bloqués ;
5. L'échange est réalisé puis marqué comme terminé ; les crédits sont transférés à l'enseignant ;
6. L'apprenant laisse un avis sur l'enseignant.

1.4 Objectifs techniques

- Concevoir l'application selon une **architecture microservices** : sept services indépendants, une base de données par service ;
- Exposer un **point d'entrée unique** (API Gateway) et déléguer l'authentification à **Keycloak** (OAuth2 / OpenID Connect, jetons JWT) ;
- Combiner **communications synchrones** (OpenFeign) et **asynchrones** (RabbitMQ) avec des mécanismes de résilience ;
- Documenter chaque API (OpenAPI / Swagger) et conteneuriser l'ensemble avec Docker Compose.

2 Acteurs et fonctionnalités

2.1 Acteurs

Tableau 1 – Acteurs de la plateforme

Acteur	Description	Actions principales
Visiteur	Personne non connectée	S’inscrire et se connecter.
Membre	Utilisateur connecté, il enseigne et apprend	Gérer son profil, publier et rechercher des compétences, demander et gérer des échanges, consulter son solde, noter un enseignant, lire ses notifications, consulter les recommandations.
Administrateur	Modérateur de la plateforme	Gérer les catégories, modérer compétences et avis, ajuster un solde, consulter les statistiques.

2.2 Diagramme de cas d’utilisation

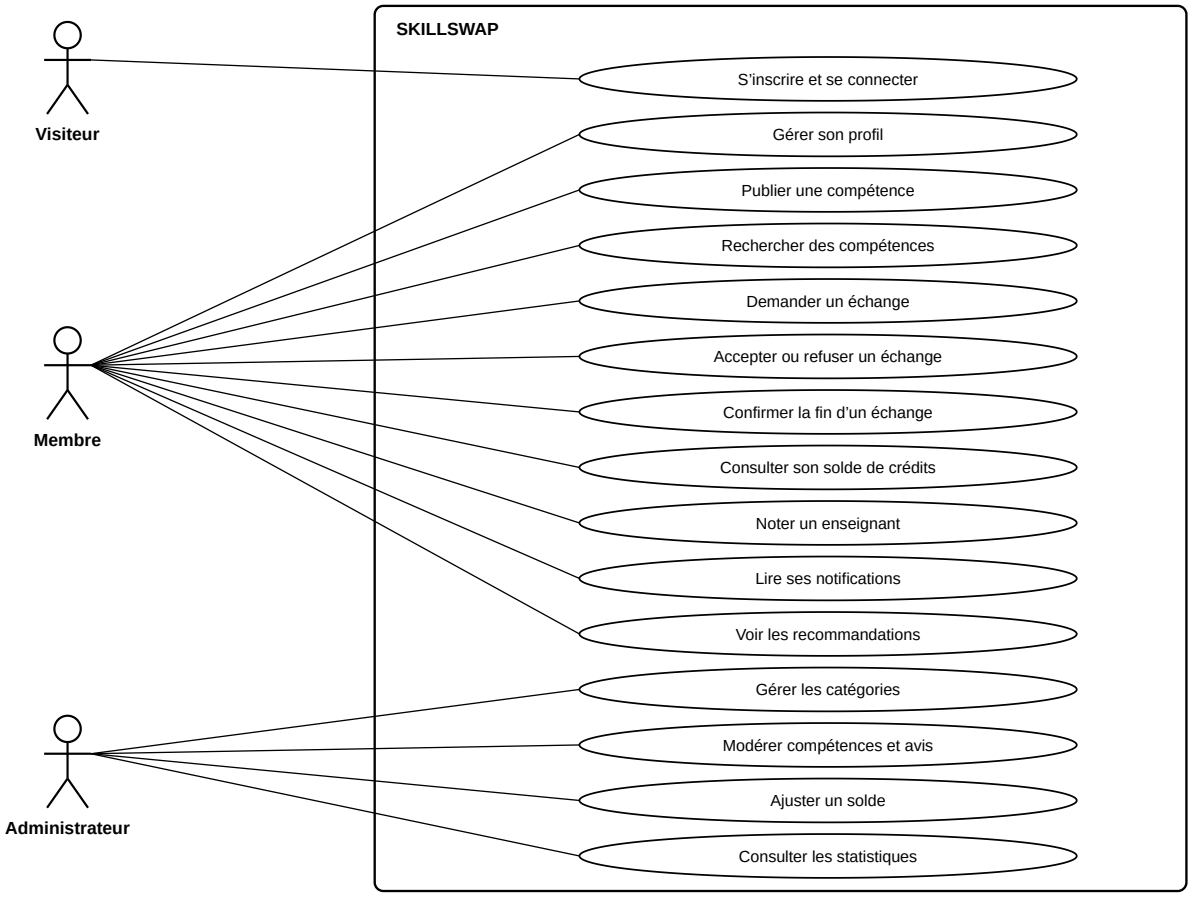


Figure 1 – Diagramme de cas d’utilisation général de SkillSwap

2.3 Fonctionnalités principales

Tableau 2 – Fonctionnalités et microservices responsables

Fonctionnalité	Acteur	Microservice
Inscription et connexion (Keycloak)	Visiteur, Membre, Administrateur	user-service
Gérer son profil	Membre	user-service
Publier et rechercher des compétences	Membre	skill-service
Demander, accepter, refuser, annuler, terminer un échange	Membre	exchange-service
Portefeuille de crédits-temps (1 heure = 1 crédit)	Membre	credit-service
Noter un enseignant et consulter sa note moyenne	Membre	review-service
Notifications dans l'application et par e-mail	Membre	notification-service
Recommandations de compétences et d'enseignants	Membre	recommendation-service
Gérer les catégories, modérer, ajuster un solde, statistiques	Administrateur	skill, review, credit, recommendation

3 Architecture globale

3.1 Vue d’ensemble

L’application repose sur un **point d’entrée unique** (l’API Gateway), sept microservices métier et une infrastructure commune (découverte de services, configuration centralisée, serveur d’identité, bus de messages). Chaque microservice possède sa propre base de données ; aucun service n’accède directement à la base d’un autre.

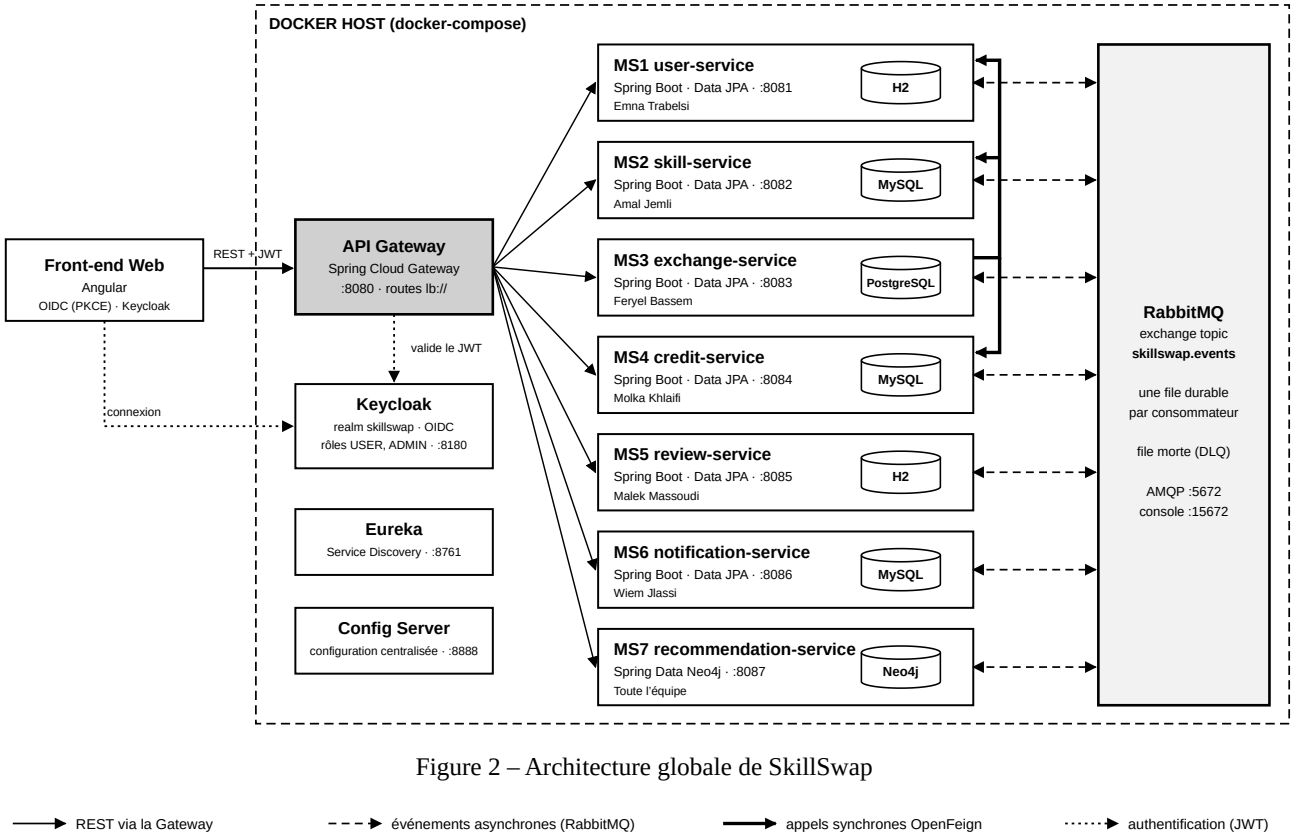


Figure 2 – Architecture globale de SkillSwap

3.2 Découpage en microservices

Tableau 3 – Microservices, ports, bases de données et responsables

Service	Responsabilité	Port	Base	Responsable
MS1 user-service	Profils des membres (l'identité reste dans Keycloak)	8081	H2	Emna Trabelsi
MS2 skill-service	Catalogue des compétences et compétences déclarées par chaque membre	8082	MySQL	Amal Jemli
MS3 exchange-service	Cycle de vie d'un échange : demande, acceptation, refus, annulation, fin	8083	PostgreSQL	Feryel Bassem
MS4 credit-service	Portefeuille de crédits-temps : solde, blocage, transfert, remboursement	8084	MySQL	Molka Khlaifi
MS5 review-service	Avis (note de 1 à 5) après un échange terminé, note moyenne	8085	H2	Malek Massoudi
MS6 notification-service	Notifications dans l'application et e-mails, déclenchées par les événements	8086	MySQL	Wiem Jlassi
MS7 recommendation-service	Service commun : graphe membres-compétences, recommandations	8087	Neo4j	Toute l'équipe

Justification du découpage. Chaque microservice correspond à une capacité métier (contexte borné) et non à une table ou à un écran :

- **user-service** : identité fonctionnelle du membre ; **skill-service** : ce que le membre sait enseigner ou veut apprendre ;
- **exchange-service** : la relation entre deux membres autour d'une compétence (machine à états de l'échange) ;
- **credit-service** : la monnaie interne (solde, blocage, transfert), isolée pour garantir l'intégrité financière ;
- **review-service** : la réputation, déclenchée seulement après un échange terminé ;
- **notification-service** : la diffusion d'information, entièrement pilotée par les événements des autres services ;
- **recommendation-service** (service commun, technologie avancée) : le graphe de relations, qui n'appartient à aucun autre domaine.

3.3 Composants d'infrastructure

Tableau 4 – Composants d'infrastructure

Composant	Technologie	Port	Rôle
API Gateway	Spring Cloud Gateway	8080	Point d'entrée unique, routage des requêtes (routes lb://), CORS, première vérification du JWT
Service Discovery	Eureka	8761	Enregistrement et découverte des instances de microservices
Config Server	Spring Cloud Config	8888	Configuration centralisée des services
Serveur d'identité	Keycloak (realm skillswap)	8180	Authentification OAuth2 / OIDC, émission des jetons JWT, rôles USER et ADMIN
Bus de messages	RabbitMQ	5672 / 15672	Exchange topic skillswap.events, une file durable par consommateur, file morte (DLQ)
Front-end	Angular	–	Interface web ; connexion OIDC (Authorization Code + PKCE) ; consomme les API via la Gateway

3.4 Principes d’architecture

- **Une base de données par service** : aucune clé étrangère entre services ; les références vers un autre service sont des *identifiants logiques* (par exemple userId) ;
- **Accès externe uniquement via la Gateway** : les services ne sont pas exposés directement ;
- **Découverte dynamique** : les appels inter-services utilisent le nom du service enregistré dans Eureka ;
- **Cohérence à terme** : les données dérivées (graphe de recommandation, notifications) sont alimentées par des événements ;
- **Contrat d’abord** : chaque API est décrite par un contrat OpenAPI avant son implémentation.

3.5 Données gérées par chaque service et choix des bases

Chaque service est propriétaire de ses données dans une base qui lui est propre. Aucune base n’est partagée et aucune clé étrangère ne relie deux services : les références vers un autre service sont des identifiants logiques, cohérents grâce aux événements.

Tableau 5 – Données gérées, base de données et références logiques par service

Service	Données gérées	Base	Justification du choix	Références logiques
user-service	Profils : username, e-mail, ville, biographie, langues	H2	Schéma simple, faible volume, démarrage rapide en développement	keycloakId (identité Keycloak)
skill-service	Catégories, compétences, compétences déclarées (type, niveau)	MySQL	Données relationnelles avec contraintes d’unicité	userId (user-service)
exchange-service	Échanges : participants, compétence, statut, dates	PostgreSQL	Intégrité des transitions d’état, historique et requêtes transactionnelles	requesterId, receiverId, skillId
credit-service	Portefeuilles, transactions, blocages de crédits (holds)	MySQL	Transactions ACID et verrou optimiste (@Version) sur les soldes	userId, exchangeId
review-service	Avis (note, commentaire), échanges éligibles	H2	Schéma simple, faible volume	exchangeId, reviewerId, revieweeId, skillId
notification-service	Notifications, préférences, envois d’e-mails, événements traités, contacts	MySQL	Volume croissant, filtres et pagination par utilisateur	userId, relatedId
recommendation-service	Graphe : Member, Skill, Category et liens OFFERS, LEARNED, RATED	Neo4j	Parcours de relations en plusieurs sauts (recommandation par voisinage)	userId, skillId

4 Technologies choisies

Tableau 6 – Technologies par domaine

Domaine	Technologie	Usage
Microservices	Spring Boot 3, Java 17	Spring Web, Spring Data JPA, Bean Validation
API Gateway	Spring Cloud Gateway	Point d’entrée unique, routage
Découverte	Eureka	Enregistrement et découverte des services
Configuration	Spring Cloud Config	Configuration centralisée
Sécurité	Keycloak	OAuth2, OIDC, JWT, rôles
Synchrone	OpenFeign, Resilience4j	Appels REST inter-services, délai de 2 s, circuit breaker
Asynchrone	RabbitMQ	Événements, files durables, file morte
Bases de données	H2, MySQL, PostgreSQL, Neo4j	Une base par service
Front-end	Angular	Consomme les API via la Gateway
Documentation	Swagger (springdoc-openapi)	Contrat OpenAPI par service
Conteneurisation	Docker, Docker Compose	Images et lancement global
Gestion de projet	Git, GitHub Projects	Branches, Pull Requests, tableau Kanban

5 Sécurité

L’authentification est entièrement déléguée à **Keycloak** : aucun mot de passe n’est stocké dans les microservices.

- **Connexion** : le front Angular utilise le flux Authorization Code avec PKCE ; Keycloak renvoie un jeton JWT ;
- **Transport du jeton** : le JWT est envoyé dans l’en-tête `Authorization: Bearer` ; l’identifiant du membre correspond à la revendication (claim) `sub` ;
- **Rôles** : `USER` (membre) et `ADMIN` (modérateur) ;
- **Double vérification** : la Gateway valide d’abord le jeton, puis chaque microservice vérifie à son tour le JWT et les rôles ;
- **Propriété des données** : un membre ne peut modifier ou supprimer que ses propres ressources, sinon le service répond **403 Forbidden** ;
- **Appels inter-services** : le jeton est propagé par OpenFeign.

6 Communications entre microservices

6.1 Communications synchrones (OpenFeign)

Les appels synchrones sont réservés aux vérifications nécessaires *avant* d'enregistrer une donnée. Ils sont protégés par Resilience4j : **délai maximal de 2 secondes** et **circuit breaker**. En cas d'échec, le service appelant répond **503 Service Unavailable** et n'enregistre rien.

Tableau 7 – Appels synchrones inter-services

Appelant	Appelé	Objet	Requête
exchange-service	user-service	Vérifier l'existence du destinataire	`GET /api/users/{id}`
exchange-service	skill-service	Vérifier la compétence proposée	`GET /api/user-skills/{id}`
exchange-service	credit-service	Bloquer les crédits de l'apprenant (escrow)	`POST /holds`

6.2 Communications asynchrones (RabbitMQ)

- **Exchange topic** unique : `skillswap.events` ; les clés de routage suivent le motif `domaine.action` ;
- **Une file durable par consommateur**, liée aux clés de routage qui l'intéressent ;
- **Acquittement manuel** (ack) après traitement réussi ;
- **Idempotence** : chaque événement porte un `eventId` ; un événement déjà traité est ignoré ;
- **Fiabilité** : 3 tentatives, puis envoi dans la file morte (DLQ).

6.3 Catalogue des événements

Tableau 8 – Événements échangés via RabbitMQ

Clé de routage	Producteur	Consommateurs	Effet chez le consommateur
`user.registered`	user-service	credit, notification, recommendation	Création du portefeuille et crédits de bienvenue ; notification de bienvenue ; nœud Member
`user.deleted`	user-service	credit, skill, notification, recommendation, review	Clôture du portefeuille ; désactivation des compétences ; suppression du contact, du nœud et des avis
`userskill.created / deleted`	skill-service	recommendation	Ajout ou retrait du lien OFFERS dans le graphe
`exchange.requested`	exchange-service	notification	« Nouvelle demande d'échange » (enseignant)
`exchange.accepted`	exchange-service	notification	« Votre échange a été accepté » (apprenant)
`exchange.rejected`	exchange-service	notification	« Votre demande a été refusée » (apprenant)
`exchange.cancelled`	exchange-service	notification, credit	Notification ; remboursement de l'apprenant
`exchange.completed`	exchange-service	credit, notification, review, recommendation	Transfert des crédits ; demande d'avis ; échange éligible à un avis ; lien LEARNED
`credit.transferred`	credit-service	notification	« Vous avez gagné X crédits »
`credit.refunded`	credit-service	notification	« X crédits vous ont été remboursés »

Clé de routage	Producteur	Consommateurs	Effet chez le consommateur
`credit.low-balance`	credit-service	notification	« Solde faible »
`review.created`	review-service	notification, skill, recommendation	« Nouvel avis reçu » ; note moyenne ; lien RATED
`review.deleted`	review-service	skill	Recalcul de la note moyenne

6.4 Gestion des erreurs

Tous les services renvoient leurs erreurs au format **Problem Details (RFC 7807)**, complété par une table `errors` qui désigne le champ en cause (message affiché sous le champ du formulaire).

Tableau 9 – Codes de réponse HTTP utilisés

Code	Signification	Exemple dans SkillSwap
200 OK	Succès	Consultation, modification
201 Created	Ressource créée	Création d’un profil, d’une demande d’échange
202 Accepted	Traitement asynchrone accepté	Annonce diffusée à tous les membres
204 No Content	Succès sans contenu	Suppression
400 Bad Request	Requête invalide	Contrôle de saisie non respecté
401 Unauthorized	Non authentifié	JWT absent ou invalide
402 Payment Required	Solde insuffisant	Blocage de crédits refusé
403 Forbidden	Accès interdit	Ressource d’un autre membre, rôle ADMIN requis
404 Not Found	Ressource introuvable	Identifiant inexistant
409 Conflict	Conflit	Username ou e-mail déjà utilisé, statut incompatible, avis déjà déposé
503 Service Unavailable	Service appelé indisponible	Délai de 2 s dépassé ou circuit ouvert

7 Conception détaillée des microservices

Ce chapitre présente, pour chaque microservice, sa description et ses responsabilités, sa fiche technique, son diagramme de classes, son contrat d'API (principaux endpoints et codes de réponse), ses événements, ses interactions avec les autres services et des exemples de communications synchrone et asynchrone sous forme de diagrammes de séquence. Les routes sont données telles qu'elles sont exposées par la Gateway.

7.1 MS1 · user-service

Description et responsabilités

Rôle. Gère les profils des membres de SkillSwap (nom, ville, biographie, langues). L'authentification reste dans Keycloak : aucun mot de passe n'est stocké dans ce service.

- Créer le profil à l'inscription (contrôle de saisie, username et e-mail uniques) ;
- Consulter son profil (`/me`), un profil par identifiant, ou rechercher avec pagination ;
- Modifier un profil (propriétaire uniquement) ;
- Supprimer un profil (propriétaire ou ADMIN) ;
- Publier `UserRegistered` et `UserDeleted` pour les autres services ;
- Renvoyer des erreurs claires (400, 404, 409) au format Problem Details.

Tableau 10 – Fiche technique de user-service

Élément	Valeur
Nom Eureka	USER-SERVICE
Port	8081
Base de données	H2 privée (aucune clé étrangère vers un autre service)
Messagerie	RabbitMQ (producteur)
Sécurité	JWT Keycloak (USER, ADMIN)
Accès	via la Gateway : /api/users/**
Documentation	Swagger (springdoc-openapi)
Technologies	Spring Boot 3.3.5, Java 17, Spring Data JPA, Bean Validation, Maven

Diagramme de classes – user-service

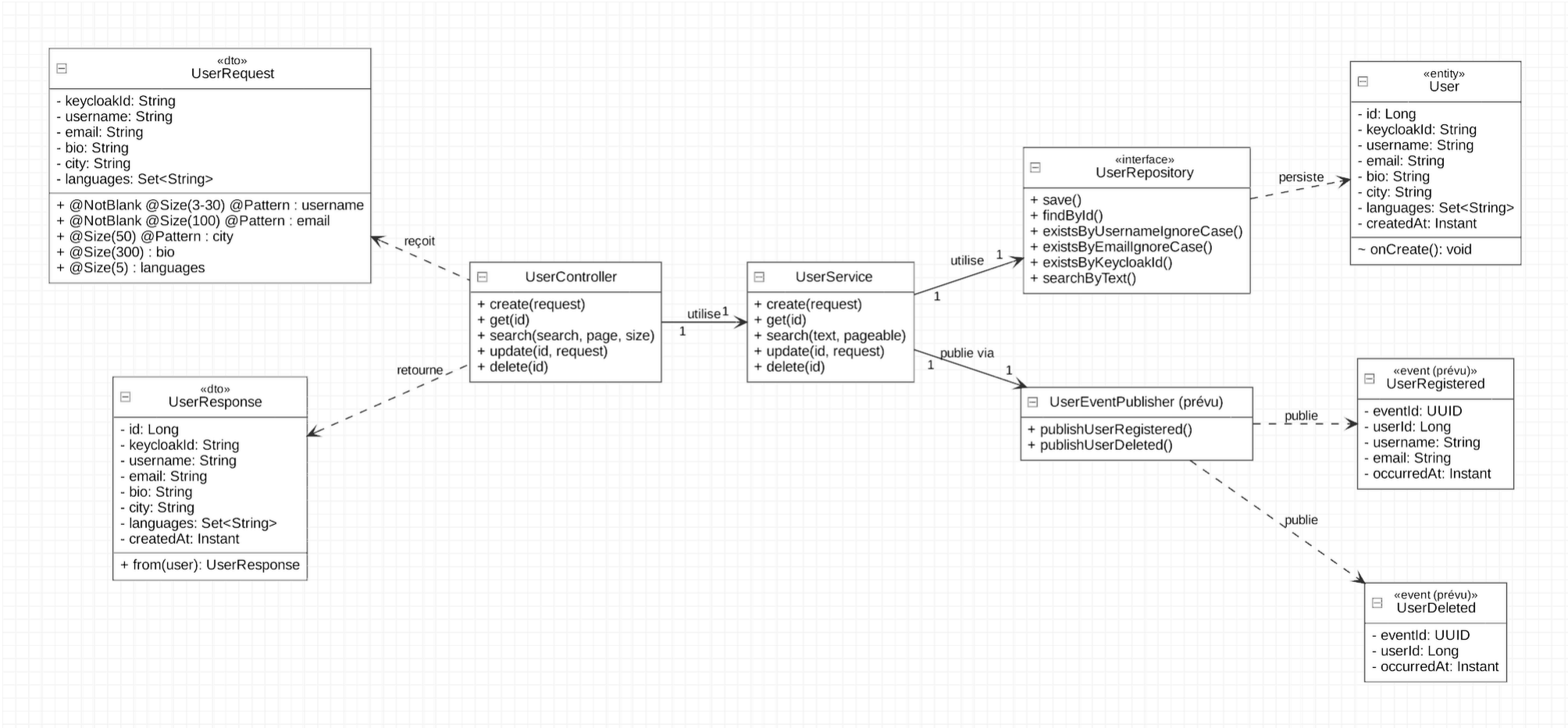


Figure 3 – Diagramme de classes de user-service

Contrat d'API

Via l'API Gateway (port 8080) ; authentification par JWT Keycloak ; rôles USER et ADMIN.

Tableau 11 – Principaux endpoints de user-service

Méthode	Endpoint	Description	Codes HTTP
POST	`/api/users`	Créer le profil (inscription)	201, 400, 409
GET	`/api/users/me`	Récupérer mon profil	200, 401, 404
GET	`/api/users/{id}`	Récupérer un utilisateur	200, 404
GET	`/api/users?search=&page=&size=`	Rechercher (paginé)	200, 400
PUT	`/api/users/{id}`	Modifier (propriétaire uniquement)	200, 400, 403, 404, 409
DELETE	`/api/users/{id}`	Supprimer (propriétaire ou ADMIN)	204, 403, 404

Codes : 200 OK · 201 Created · 204 No Content · 400 requête invalide · 401 non authentifié · 403 accès interdit · 404 introuvable · 409 conflit.

Événements RabbitMQ

Tableau 12 – Événements de user-service

Sens	Événement	Détail
Consommé	–	Aucun événement consommé : user-service ne dépend d'aucun autre microservice.
Publié	`user.registered`	UserRegistered → credit-service (portefeuille et bonus), notification-service (bienvenue), recommendation-service (nœud Member)
Publié	`user.deleted`	UserDeleted → credit-service (clôture du portefeuille), skill-service (désactivation des compétences), recommendation-service (suppression du nœud), review-service et notification-service

Aucun appel synchrone sortant : seul le front appelle user-service, via la Gateway ; user-service est appelé de façon synchrone par exchange-service pour vérifier le destinataire.

Interactions avec les autres microservices

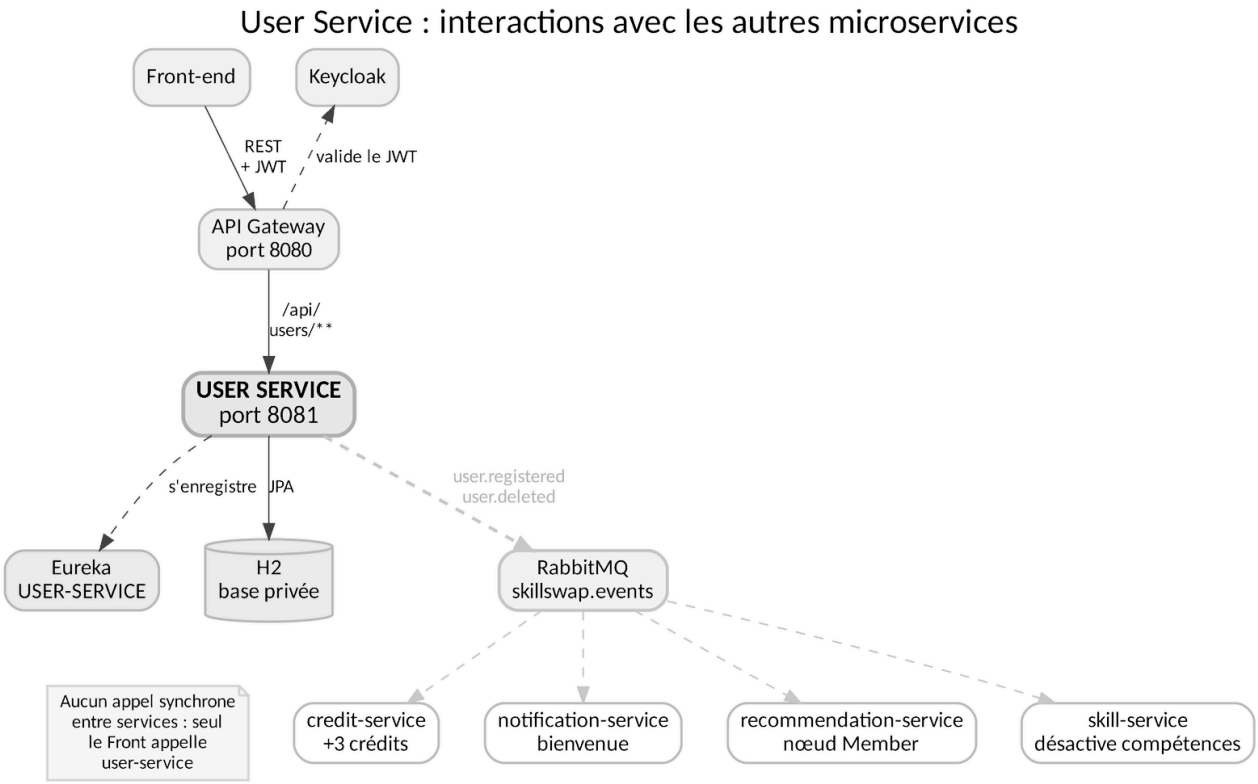


Figure 4 – Interactions de user-service avec les autres microservices

Exemple de communication synchrone

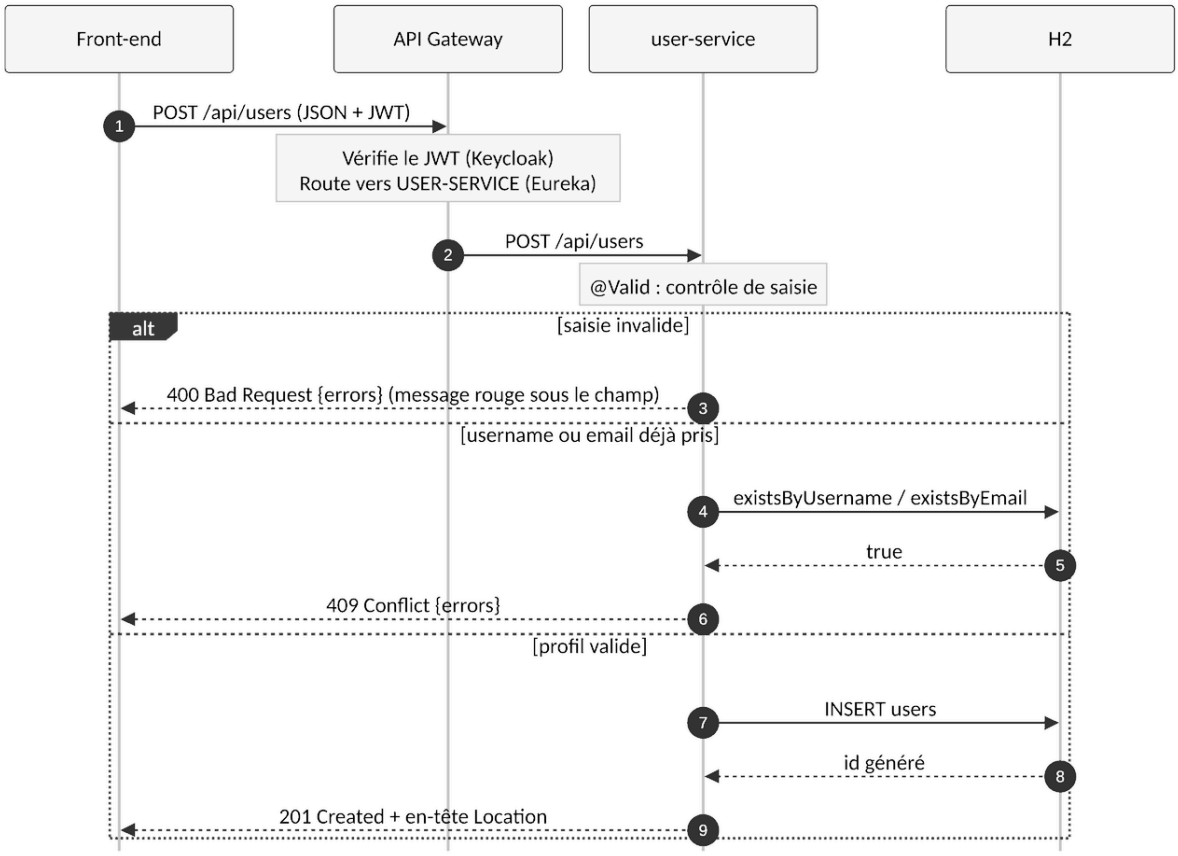


Figure 5 – Création d'un profil (POST /api/users)

Exemple de communication asynchrone

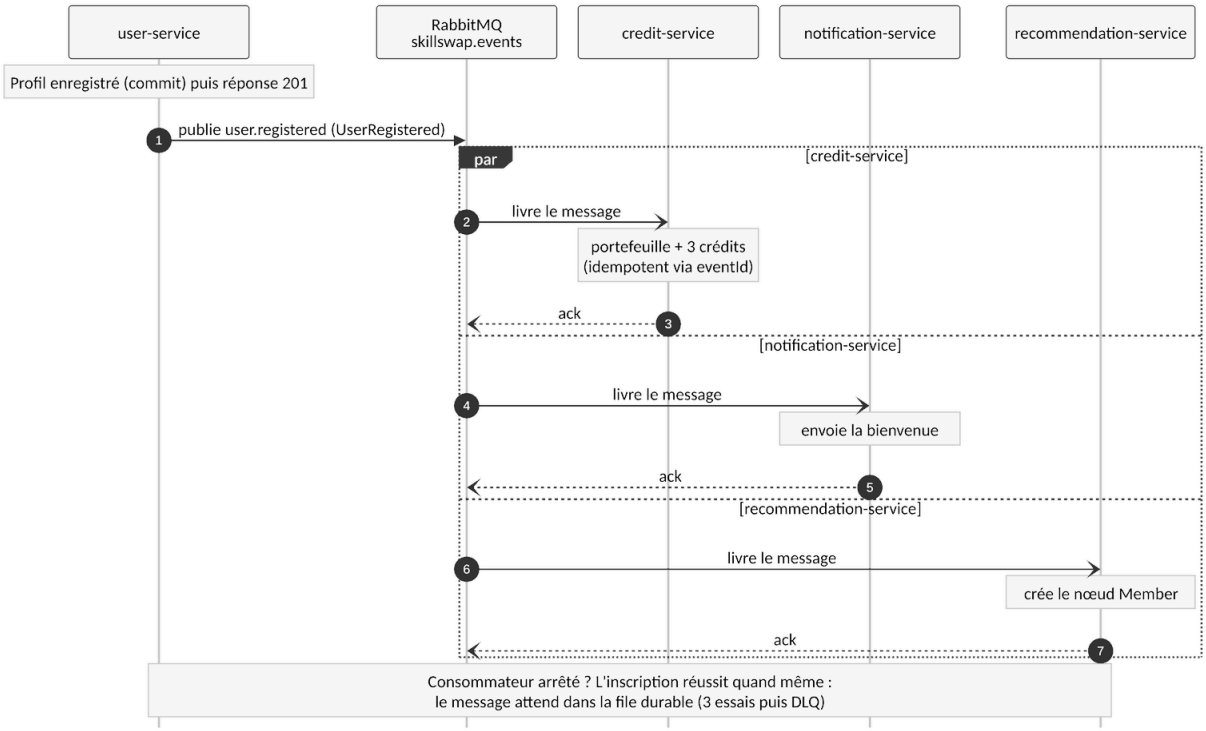


Figure 6 – Publication de user.registered et propagation aux services abonnés

État d’avancement (MS1)

Le service est développé comme projet Spring Boot autonome. Le tableau distingue ce qui est réalisé de ce qui est prévu lors de l’intégration avec le reste de la plateforme.

Tableau 13 – État d’avancement de user-service

Élément	État
CRUD complet sur les profils (création, lecture, recherche paginée, modification, suppression)	Réalisé
Contrôle de saisie (Bean Validation : username 3 à 30 caractères, e-mail, ville, biographie, langues) et unicité du username et de l’e-mail	Réalisé
Erreurs 400, 404 et 409 au format Problem Details avec table `errors`	Réalisé
Base H2 privée, documentation Swagger, CORS pour le front	Réalisé
Jeu de tests automatisés (MockMvc) – à exécuter avec `mvn test`	Rédigé
Sécurité JWT Keycloak : rôles USER / ADMIN, règle de propriété (403), endpoint `/me`	Prévu
Enregistrement Eureka et configuration centralisée	Prévu
Publication RabbitMQ de `user.registered` et `user.deleted`	Prévu

7.2 MS2 · skill-service

Description et responsabilités

Rôle. Gère le catalogue des compétences (catégories, compétences) et les compétences déclarées par chaque membre : proposées (OFFERED) ou recherchées (WANTED), avec un niveau.

- Gérer le catalogue : catégories et compétences (ADMIN), lecture publique ;
- Déclarer, modifier, supprimer ses compétences (propriétaire uniquement) ;
- Rechercher par membre, compétence ou type, avec pagination ;
- Vérifier une compétence offerte pour exchange-service (OpenFeign) ;
- Publier `UserSkillCreated` / `UserSkillDeleted`, consommer `UserDeleted` ;
- Renvoyer des erreurs claires (400, 404, 409) au format Problem Details.

Tableau 14 – Fiche technique de skill-service

Élément	Valeur
Nom Eureka	SKILL-SERVICE
Port	8082
Base de données	MySQL privée (données relationnelles et contraintes d'unicité ; aucune clé étrangère vers un autre service)
Messagerie	RabbitMQ (producteur et consommateur)
Sécurité	JWT Keycloak (USER, ADMIN)
Accès	via la Gateway : /api/skills/**, /api/categories/**, /api/user-skills/**
Documentation	Swagger

Diagramme de classes – skill-service

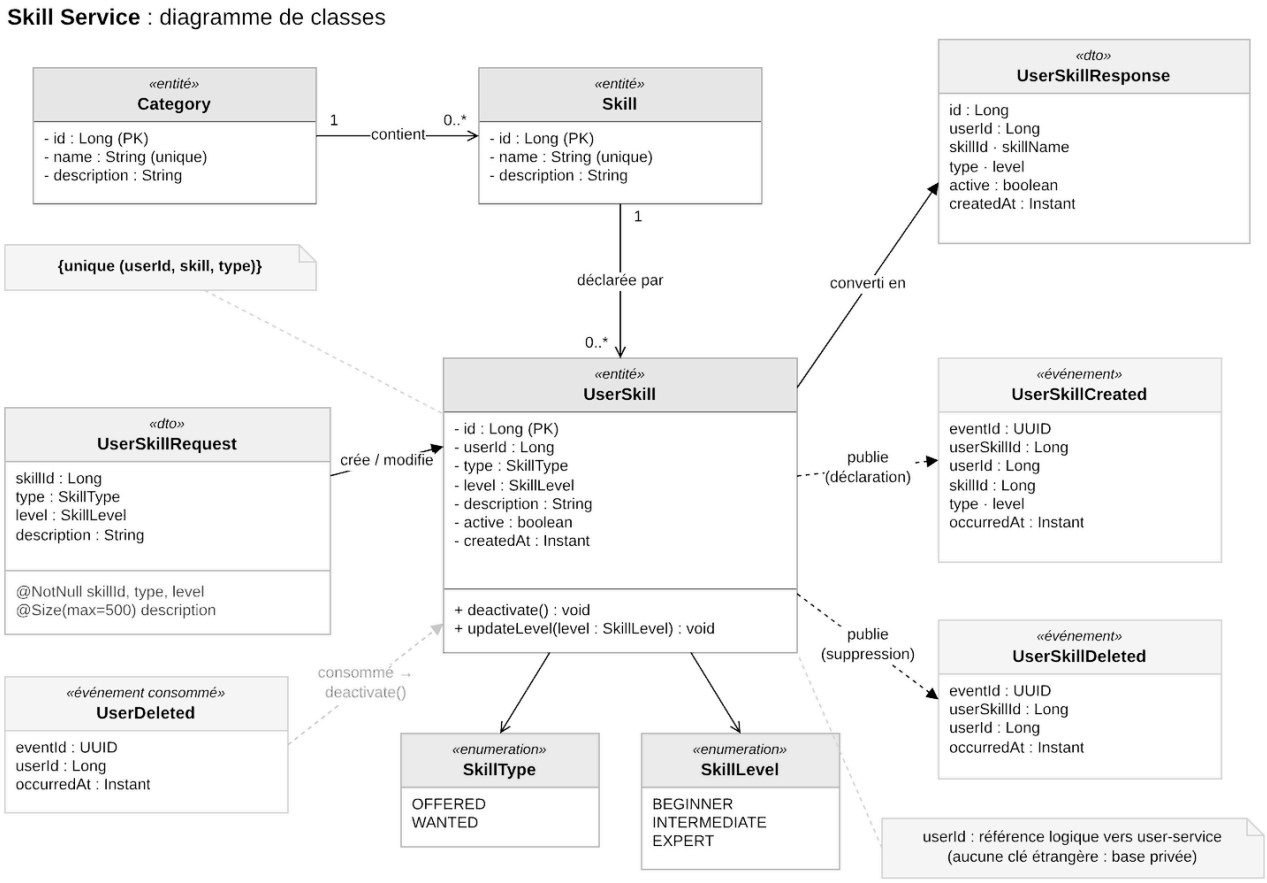


Figure 7 – Diagramme de classes de skill-service

Contrat d'API

Via l'API Gateway (port 8080) ; JWT Keycloak ; rôles USER et ADMIN ; catalogue en lecture publique.

Tableau 15 – Principaux endpoints de skill-service

Méthode	Endpoint	Description	Codes HTTP
GET	`/api/skills?categoryId=&q=`	Consulter le catalogue (public)	200
POST	`/api/skills`	Ajouter au catalogue (ADMIN)	201, 400, 401, 403, 404, 409
POST	`/api/user-skills`	Déclarer une compétence	201, 400, 401, 404, 409
GET	`/api/user-skills/me`	Consulter mes compétences	200, 401
GET	`/api/user-skills/{id}`	Consulter une compétence (exchange-service)	200, 401, 404
GET	`/api/user-skills?userId=&type=&page=&size=`	Rechercher (paginé)	200, 400, 401
PUT	`/api/user-skills/{id}`	Modifier (propriétaire uniquement)	200, 400, 401, 403, 404, 409
DELETE	`/api/user-skills/{id}`	Supprimer (propriétaire ou ADMIN)	204, 401, 403, 404

Codes : 200 OK · 201 Created · 204 No Content · 400 requête invalide · 401 non authentifié · 403 accès interdit · 404 introuvable · 409 conflit (nom existant ou compétence déjà déclarée).

Événements RabbitMQ

Tableau 16 – Événements de skill-service

Sens	Événement	Détail
Consommé	`user.deleted`	Publié par user-service ; `deactivate()` sur toutes les compétences du membre ; idempotent via eventId
Consommé	`review.created / deleted`	Publiés par review-service ; recalcul de la note moyenne de la compétence
Publié	`userskill.created`	UserSkillCreated → recommendation-service : ajoute l'offre ou le besoin au matching
Publié	`userskill.deleted`	UserSkillDeleted → recommendation-service : retire la compétence du matching

Appel synchrone entrant : exchange-service appelle `GET /api/user-skills/{id}` (OpenFeign). Aucun appel synchrone sortant : skill-service n'appelle aucun autre service.

Interactions avec les autres microservices

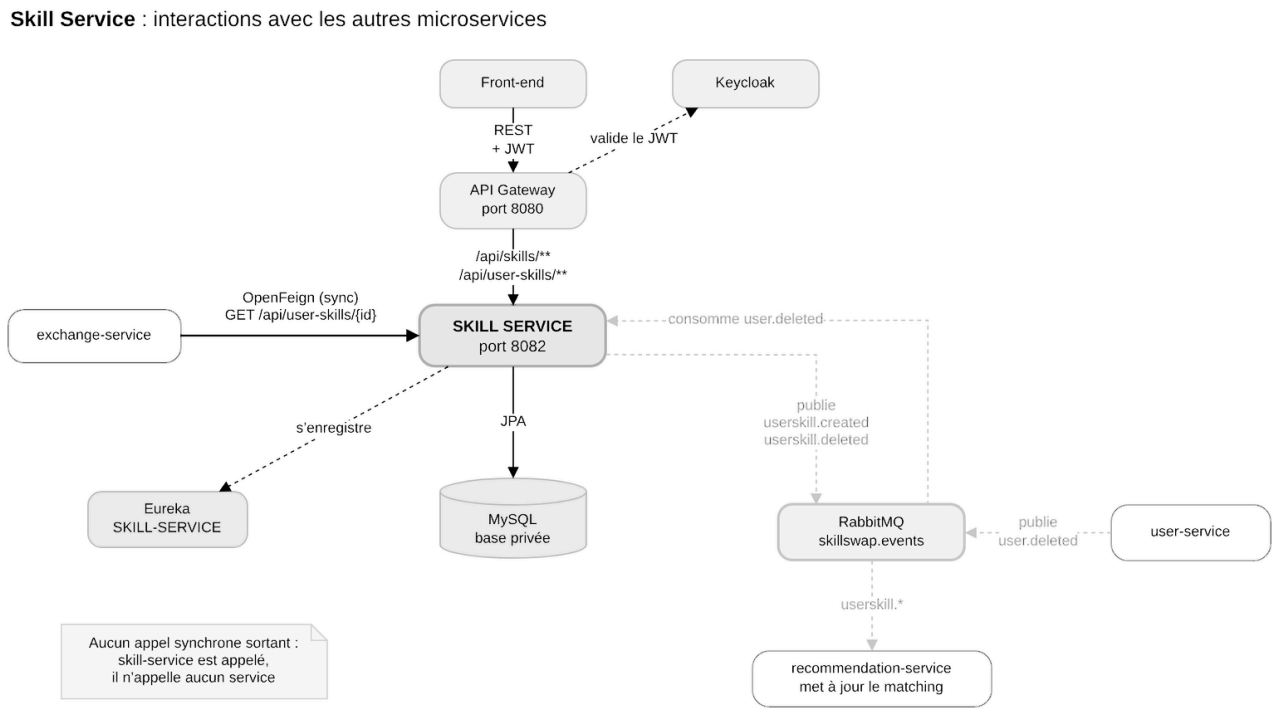


Figure 8 – Interactions de skill-service avec les autres microservices

Exemple de communication synchrone

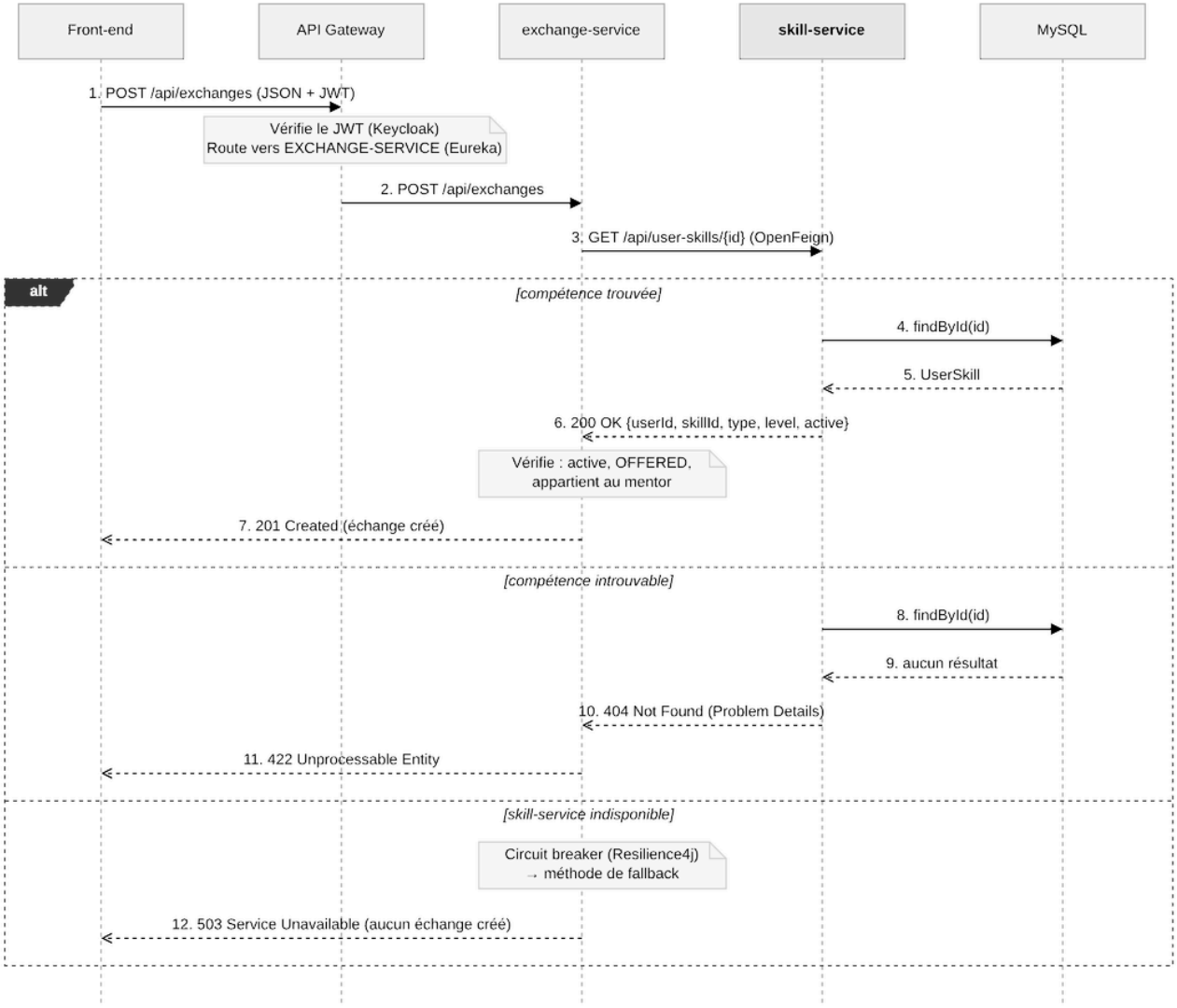


Figure 9 – Vérification d'une compétence par exchange-service (OpenFeign)

Exemple de communication asynchrone

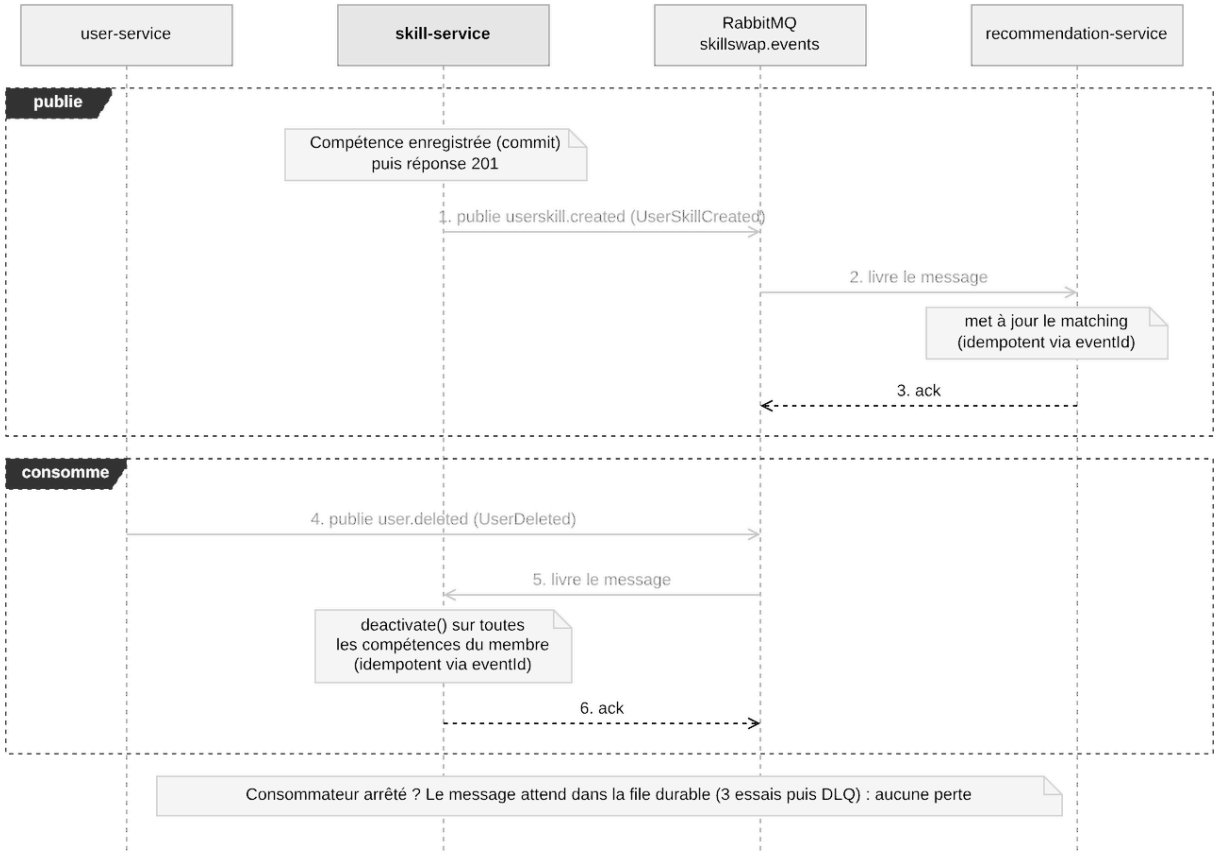


Figure 10 – Déclaration d’une compétence et consommation de user.deleted

7.3 MS3 · exchange-service

Description et responsabilités

Rôle. Gère le cycle de vie des demandes d'échange de compétences. Entité principale : **Exchange**. États : PENDING, ACCEPTED, REJECTED, CANCELLED, COMPLETED.

- Créer une nouvelle demande d'échange entre deux utilisateurs ;
- Consulter les informations et l'historique des échanges ;
- Accepter ou refuser une demande reçue ;
- Permettre l'annulation d'une demande en attente ;
- Marquer un échange comme terminé après sa réalisation ;
- Vérifier les utilisateurs et les compétences (OpenFeign) et publier des événements.

Tableau 17 – Fiche technique de exchange-service

Élément	Valeur
Nom Eureka	EXCHANGE-SERVICE
Port	8083
Base de données	PostgreSQL (base propre au service)
Persistance	Spring Data JPA
Synchrone	REST / OpenFeign avec user-service, skill-service et credit-service
Asynchrone	RabbitMQ (producteur)
Sécurité	JWT
Accès	via la Gateway : /api/exchanges/**
Technologies	Spring Boot, PostgreSQL, Spring Data JPA, OpenFeign, RabbitMQ, JWT

Diagramme de classes – exchange-service

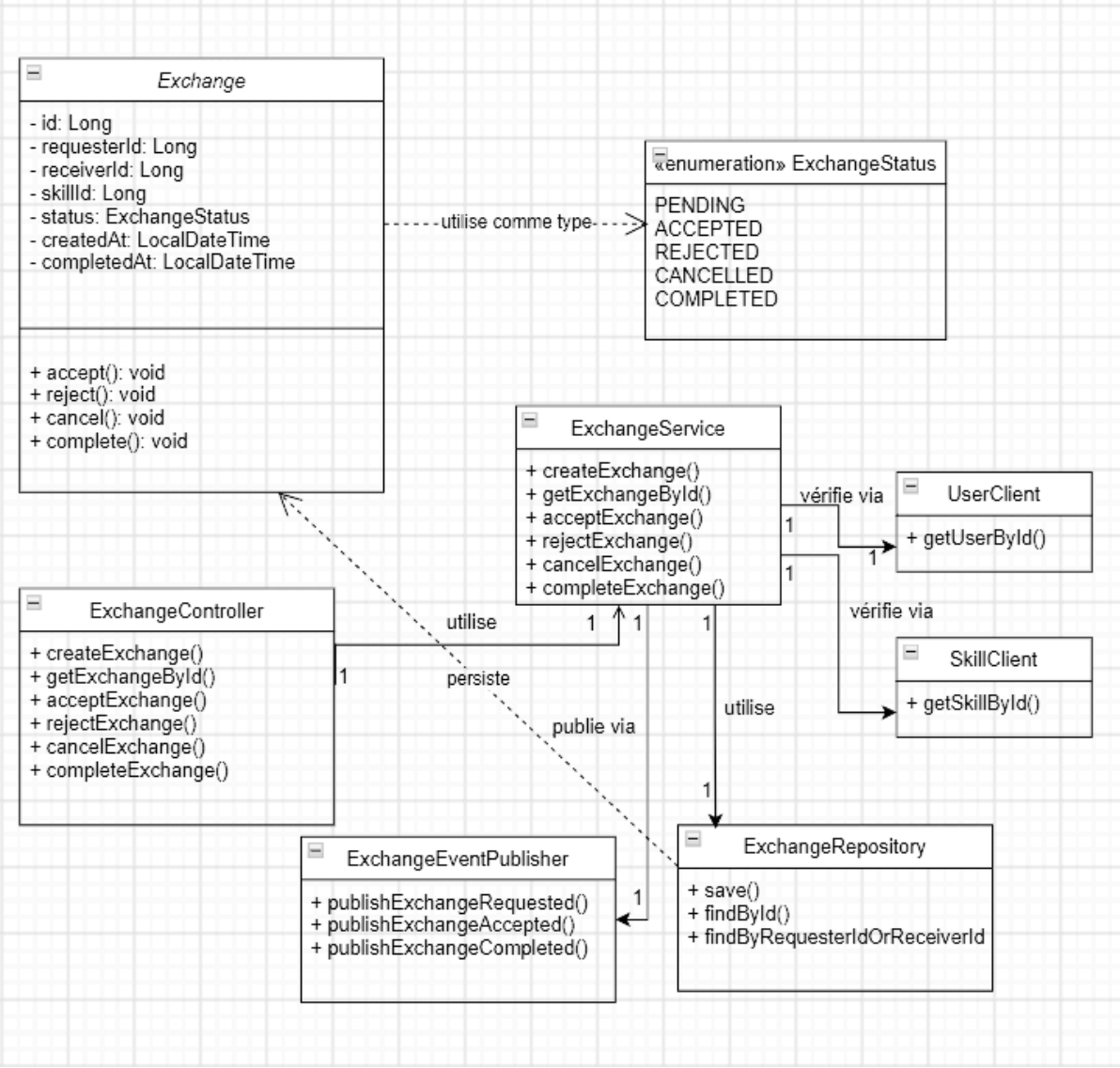


Figure 11 – Diagramme de classes d’exchange-service

Contrat d’API

Via l’API Gateway ; authentification par JWT.

Tableau 18 – Principaux endpoints de exchange-service

Méthode	Endpoint	Description	Codes HTTP
POST	`/api/exchanges`	Créer une demande	201, 400, 404, 409
GET	`/api/exchanges/{id}`	Consulter une demande	200, 403, 404
GET	`/api/exchanges/my`	Consulter mes échanges	200, 401
PATCH	`/api/exchanges/{id}/accept`	Accepter une demande	200, 403, 404, 409
PATCH	`/api/exchanges/{id}/reject`	Refuser une demande	200, 403, 404, 409
PATCH	`/api/exchanges/{id}/cancel`	Annuler une demande	200, 403, 404, 409
PATCH	`/api/exchanges/{id}/complete`	Terminer un échange	200, 403, 404, 409

Codes : 200 OK · 201 Created · 400 requête invalide · 401 non authentifié · 403 accès interdit (non-destinataire) · 404 introuvable · 409 conflit (statut incompatible).

Événements RabbitMQ

Tableau 19 – Événements de exchange-service

Sens	Événement	Détail
Consommé	–	Aucun événement consommé.
Publié	`exchange.requested`	ExchangeRequested (création d'une demande) → notification-service
Publié	`exchange.accepted`	ExchangeAccepted (acceptation) → notification-service
Publié	`exchange.rejected`	ExchangeRejected (refus) → notification-service
Publié	`exchange.cancelled`	ExchangeCancelled (annulation) → notification-service, credit-service
Publié	`exchange.completed`	ExchangeCompleted (fin de l'échange) → credit-service, notification-service, review-service, recommendation-service

Appels synchrones (OpenFeign) : **UserClient** vérifie l'existence du destinataire ; **SkillClient** vérifie la validité de la compétence ; credit-service bloque les crédits. Règles vérifiées avant enregistrement : pas d'échange avec soi-même ; la compétence appartient au destinataire.

Interactions avec les autres microservices

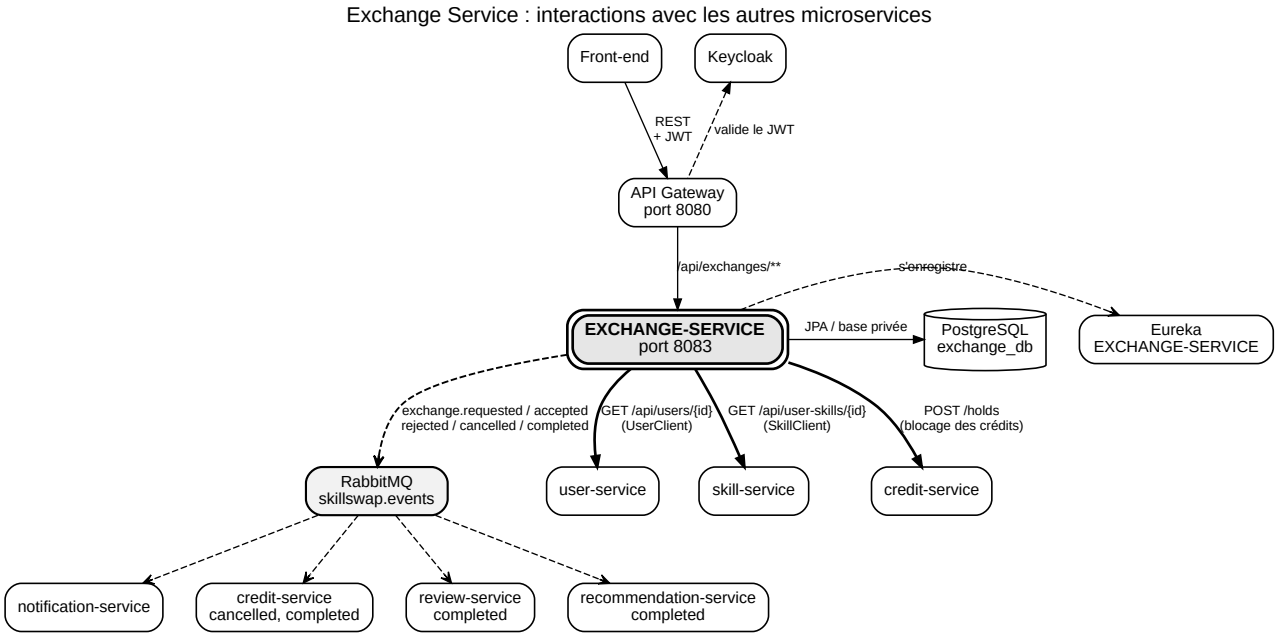


Figure 12 – Interactions de exchange-service avec les autres microservices

Exemple de communication synchrone

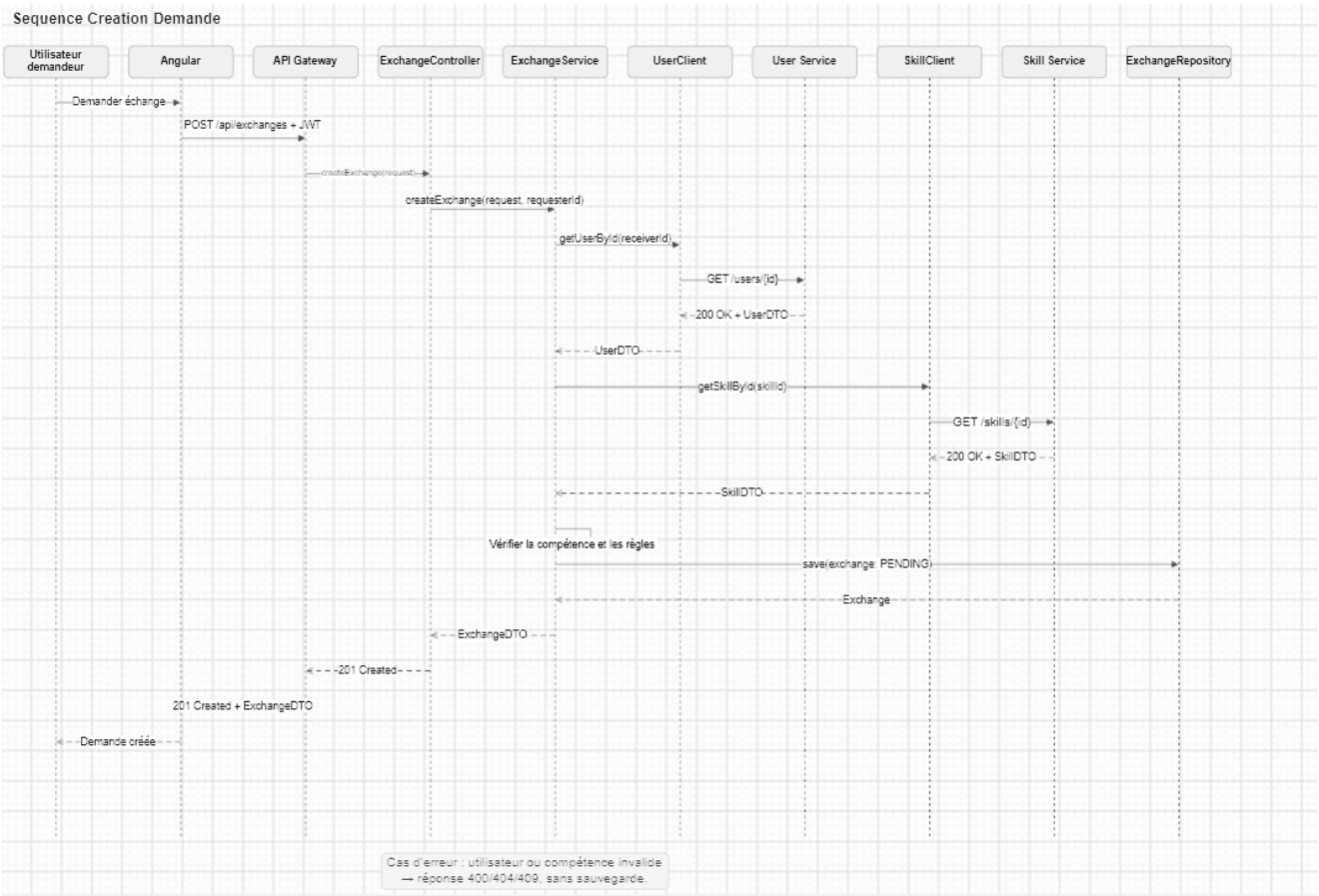


Figure 13 – Création d’une demande d’échange (REST / OpenFeign)

Exemple de communication asynchrone

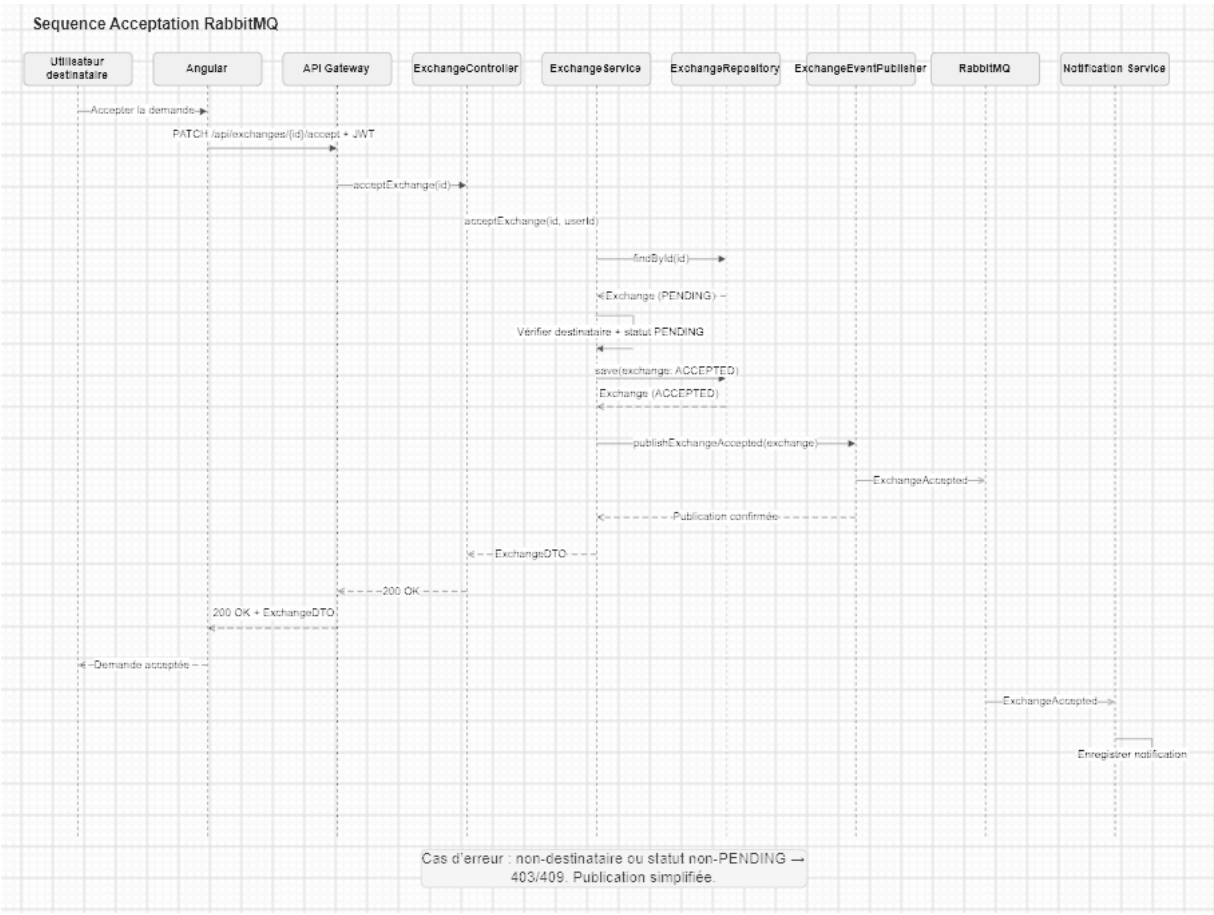


Figure 14 – Fin d'un échange : publication de exchange.completed

7.4 MS4 · credit-service

Description et responsabilités

Rôle. Gère les crédits-temps (1 crédit = 1 heure de compétence) qui remplacent l'argent sur la plateforme.

- Créer le portefeuille de chaque utilisateur (crédits de bienvenue) ;
- Consulter le solde et l'historique des transactions ;
- Bloquer les crédits de l'apprenant quand un échange est accepté (escrow) ;
- Transférer les crédits à l'enseignant quand l'échange est terminé ;
- Rembourser l'apprenant si l'échange est annulé ;
- Publier des événements pour le service de notification.

Tableau 20 – Fiche technique de credit-service

Élément	Valeur
Nom Eureka	CREDIT-SERVICE
Port	8084
Base de données	MySQL credit_db
Messagerie	RabbitMQ (producteur et consommateur)
Sécurité	JWT Keycloak (USER, ADMIN)
Accès	via la Gateway : /api/credit/**
Documentation	Swagger

Diagramme de classes – credit-service

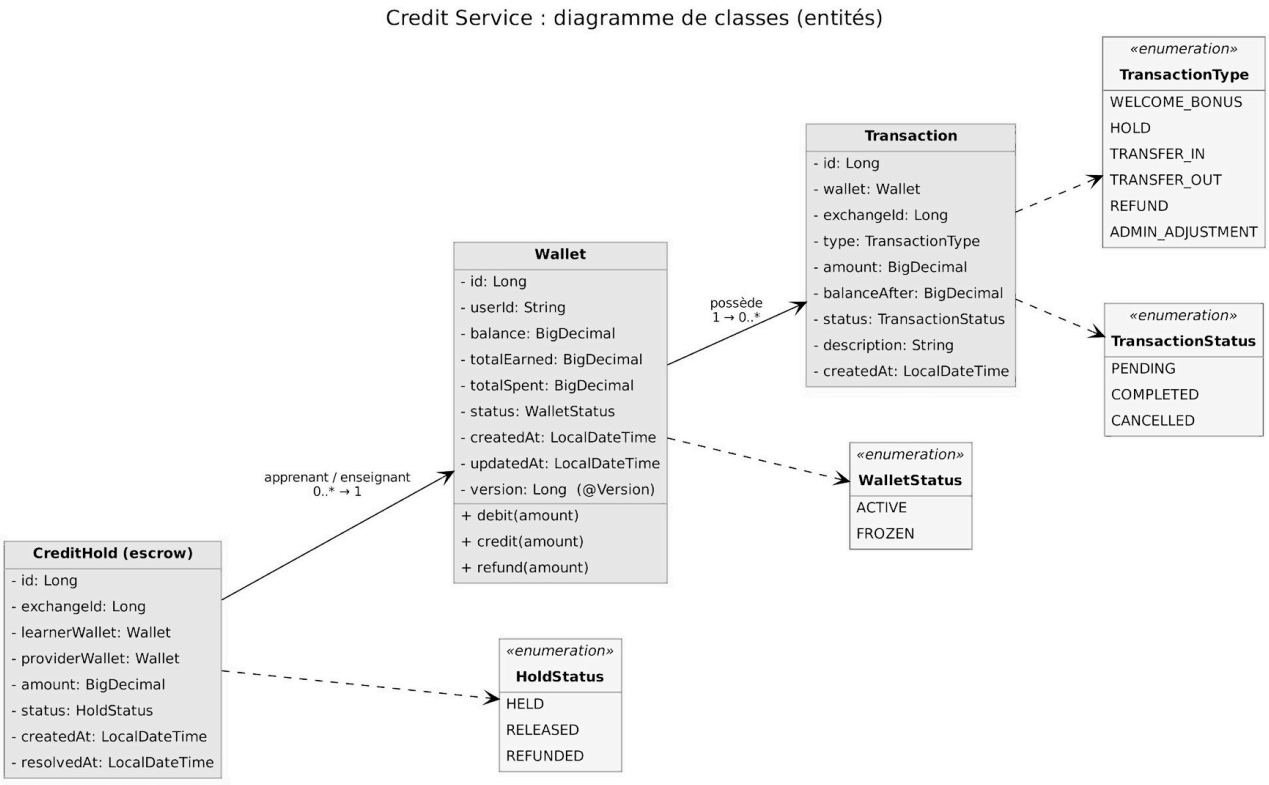


Figure 15 – Diagramme de classes de credit-service

Contrat d'API

Préfixe via l'API Gateway : `/api/credit` ; authentification par JWT.

Tableau 21 – Principaux endpoints de credit-service

Méthode	Endpoint	Description	Codes HTTP
GET	`/wallets/me`	Consulter mon solde	200, 401, 404
GET	`/wallets/me/transactions`	Mon historique paginé	200, 400, 401
POST	`/holds`	Bloquer les crédits (appel d'exchange-service)	201, 400, 402, 404, 409
POST	`/holds/{id}/release`	Transférer à l'enseignant	200, 404, 409
POST	`/holds/{id}/refund`	Rembourser l'apprenant	200, 404, 409
POST	`/admin/wallets/{id}/topup`	Ajouter des crédits (ADMIN)	200, 400, 403, 404

Codes : 200 OK · 201 Created · 400 requête invalide · 401 non authentifié · 402 solde insuffisant · 403 accès interdit · 404 introuvable · 409 conflit (hold déjà traité, wallet gelé).

Événements RabbitMQ

Tableau 22 – Événements de credit-service

Sens	Événement	Détail
Consommé	`user.registered`	User Service : création du wallet et des crédits de bienvenue
Consommé	`exchange.completed`	Exchange Service : transfert des crédits à l'enseignant
Consommé	`exchange.cancelled`	Exchange Service : remboursement de l'apprenant
Publié	`credit.transferred`	→ notification-service : « Vous avez gagné X crédits »
Publié	`credit.refunded`	→ notification-service : « X crédits vous ont été remboursés »
Publié	`credit.low-balance`	→ notification-service : « Solde faible »

Appel synchrone entrant : exchange-service appelle `POST /holds` (OpenFeign). Aucun lien direct avec skill-service, review-service et recommendation-service.

Interactions avec les autres microservices

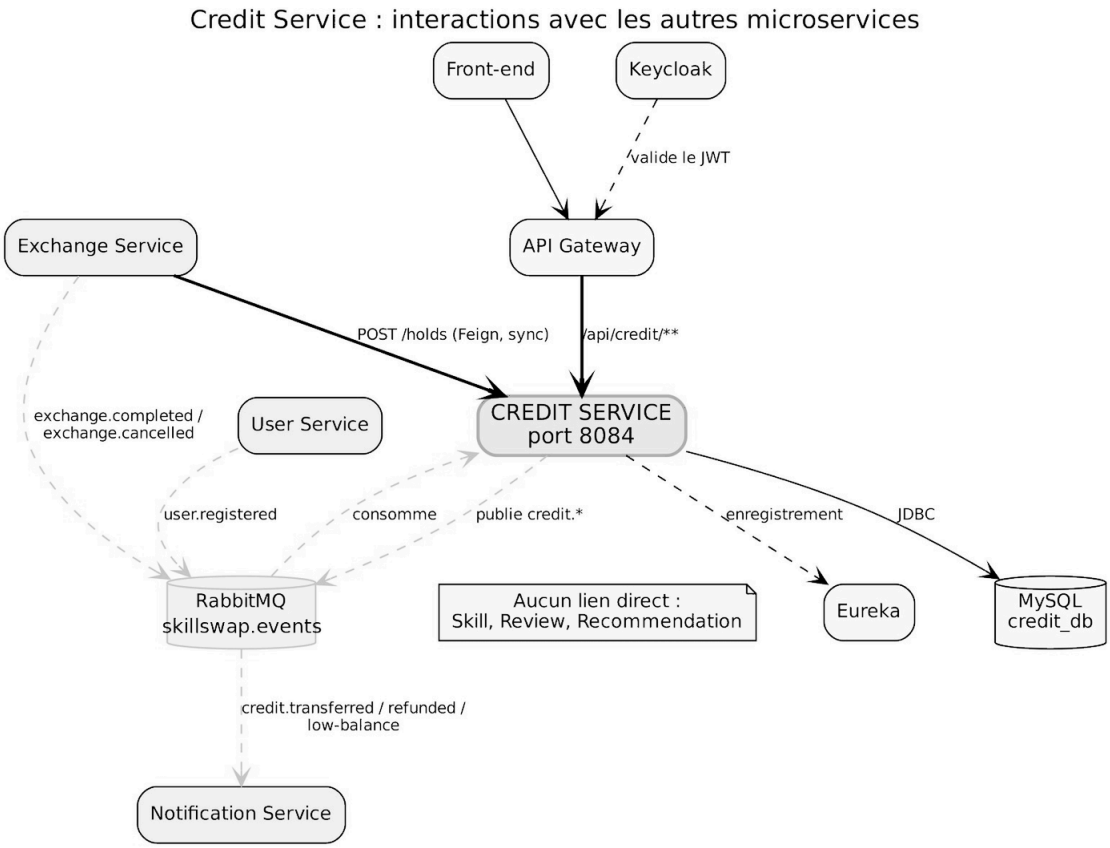


Figure 16 – Interactions de credit-service avec les autres microservices

Exemple de communication synchrone

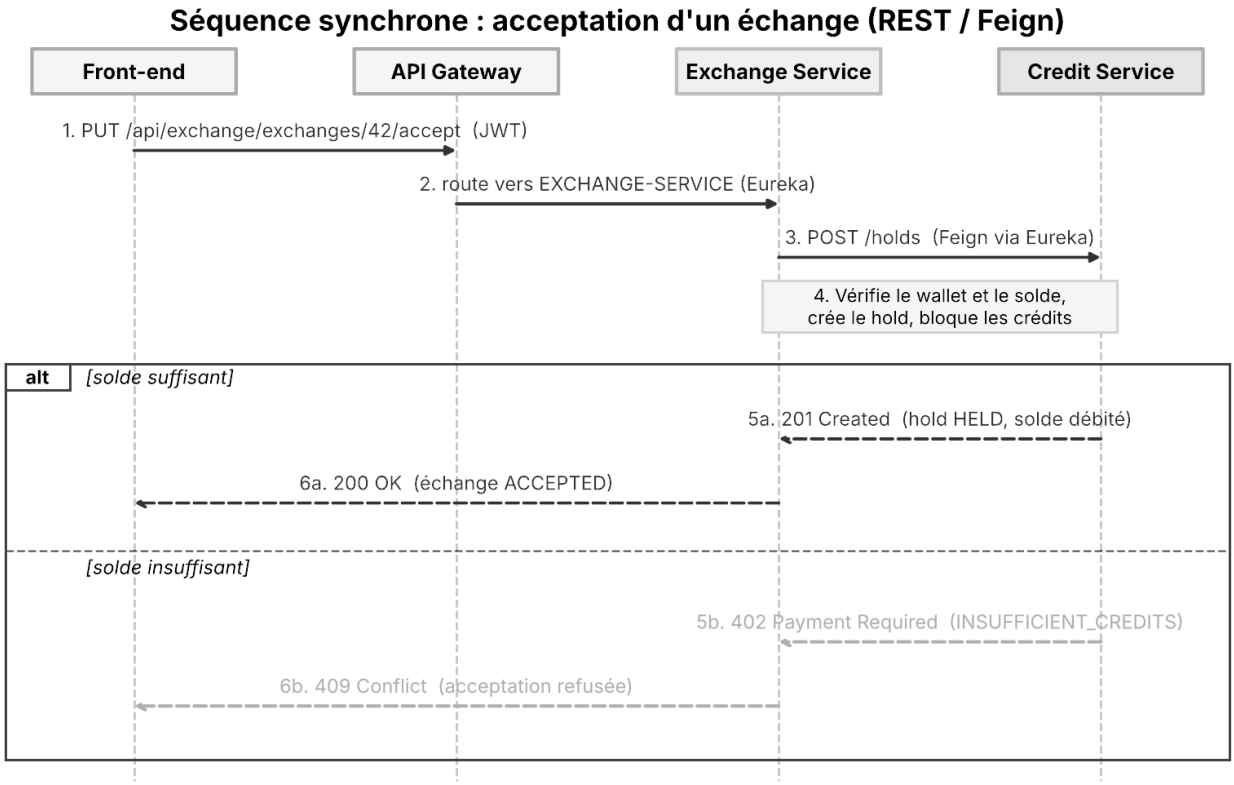


Figure 17 – Blocage des crédits (POST /holds)

Exemple de communication asynchrone

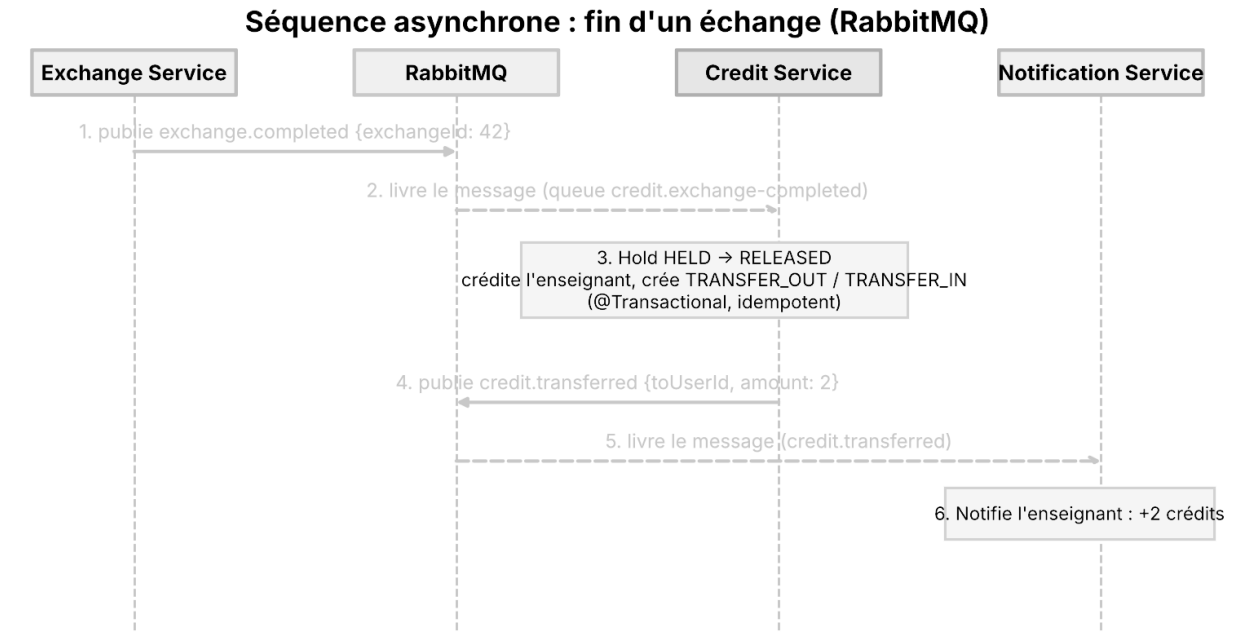


Figure 18 – Transfert des crédits après exchange.completed

7.5 MS5 · review-service

Description et responsabilités

Rôle. Gère les avis (note de 1 à 5 et commentaire) laissés par un membre sur un autre membre après un échange de compétences terminé. L'authentification reste dans Keycloak.

- Créer un avis après un échange terminé (note de 1 à 5, un seul avis par échange et par auteur) ;
- Consulter un avis par identifiant, ou lister les avis d'un membre avec pagination ;
- Calculer la note moyenne et le nombre d'avis d'un membre ;
- Modifier un avis (auteur uniquement) ; supprimer (auteur ou ADMIN) ;
- Consommer `ExchangeCompleted` et `UserDeleted` ; publier `ReviewCreated` et `ReviewDeleted` ;
- Renvoyer des erreurs claires (400, 403, 404, 409) au format Problem Details.

Tableau 23 – Fiche technique de review-service

Élément	Valeur
Nom Eureka	REVIEW-SERVICE
Port	8085
Base de données	H2 privée (aucune clé étrangère vers un autre service)
Messagerie	RabbitMQ (producteur et consommateur)
Sécurité	JWT Keycloak (USER, ADMIN)
Accès	via la Gateway : /api/reviews/**
Documentation	Swagger

Diagramme de classes – review-service

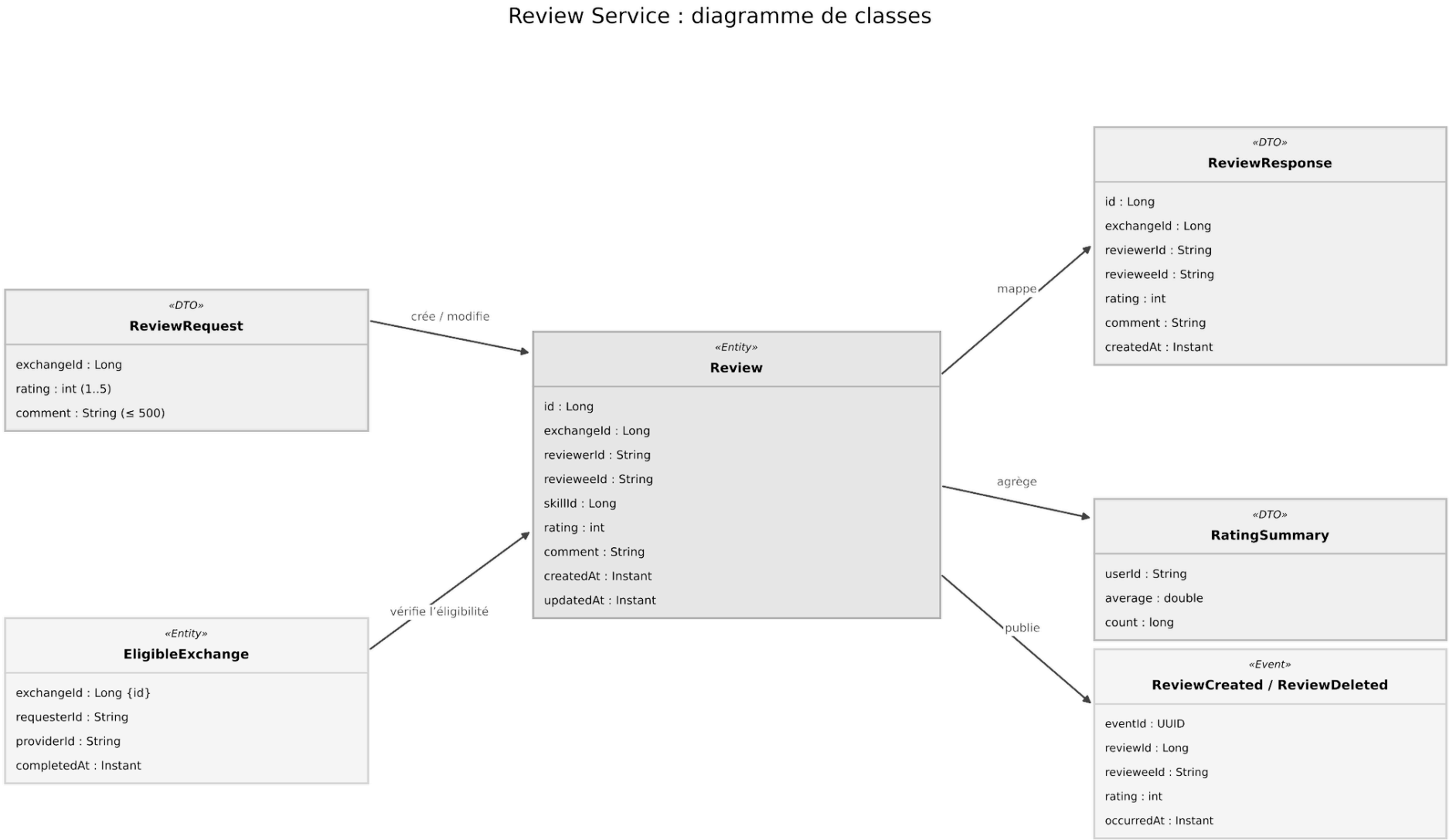


Figure 19 – Diagramme de classes de review-service

Contrat d'API

Via l'API Gateway (port 8080) ; authentification par JWT Keycloak ; rôles USER et ADMIN.

Tableau 24 – Principaux endpoints de review-service

Méthode	Endpoint	Description	Codes HTTP
POST	`/api/reviews`	Créer un avis (échange terminé)	201, 400, 403, 409
GET	`/api/reviews/{id}`	Consulter un avis	200, 404
GET	`/api/reviews?userId=&page=&size=`	Lister les avis d'un membre (paginé)	200, 400
GET	`/api/reviews/users/{userId}/average`	Note moyenne d'un membre	200, 404
PUT	`/api/reviews/{id}`	Modifier (auteur uniquement)	200, 400, 403, 404
DELETE	`/api/reviews/{id}`	Supprimer (auteur ou ADMIN)	204, 403, 404

Codes : 200 OK · 201 Created · 204 No Content · 400 requête invalide · 401 non authentifié · 403 accès interdit (échange non éligible ou non-auteur) · 404 introuvable · 409 conflit (avis déjà déposé pour cet échange).

Événements RabbitMQ

Tableau 25 – Événements de review-service

Sens	Événement	Détail
Consommé	`exchange.completed`	ExchangeCompleted (exchange-service) : l'échange devient éligible à un avis
Consommé	`user.deleted`	UserDeleted (user-service) : suppression des avis de ce membre
Publié	`review.created`	ReviewCreated → notification-service (prévient le membre noté), skill-service (note moyenne de la compétence), recommendation-service (lien RATED)
Publié	`review.deleted`	ReviewDeleted → skill-service : recalcul de la note moyenne

Aucun appel synchrone entre services : l'éligibilité d'un échange est connue grâce à l'événement exchange.completed (copie locale EligibleExchange).

Interactions avec les autres microservices

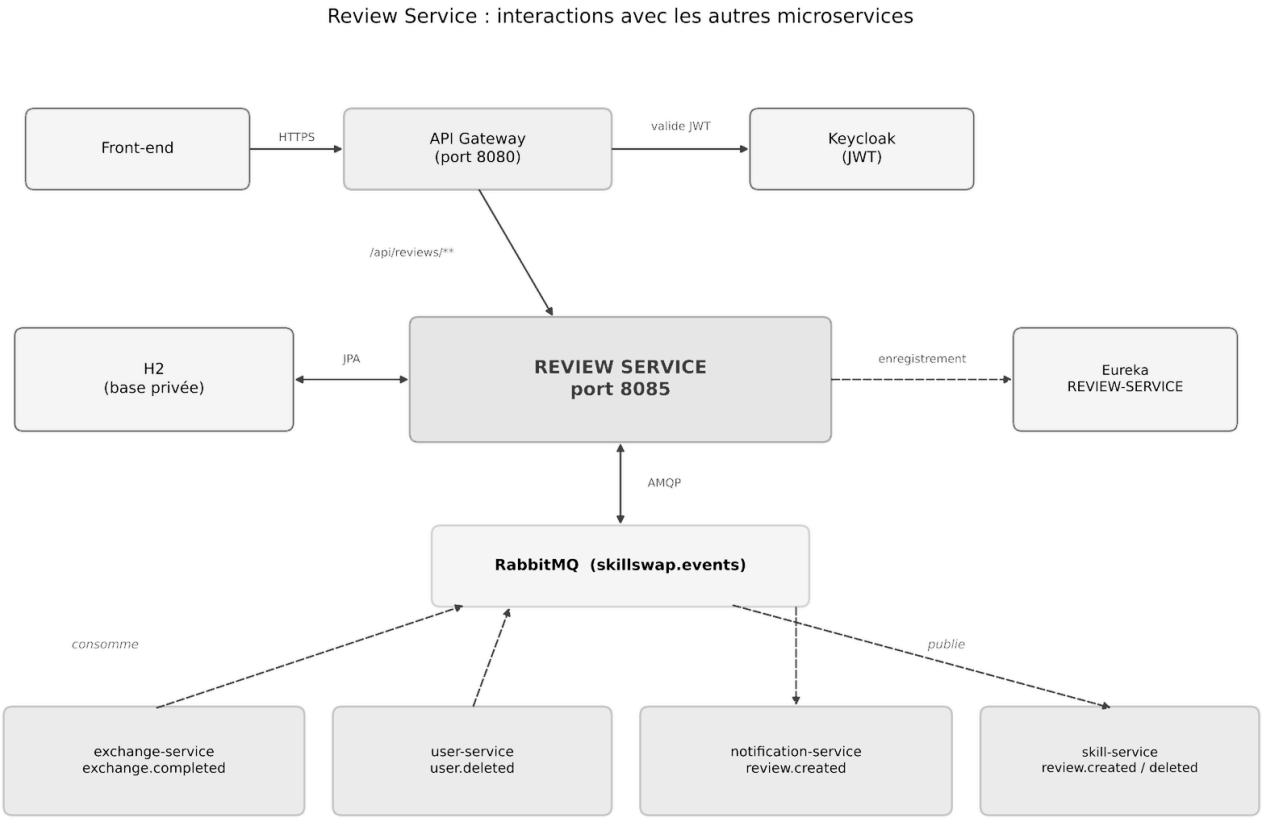


Figure 20 – Interactions de review-service avec les autres microservices

Exemple de communication synchrone

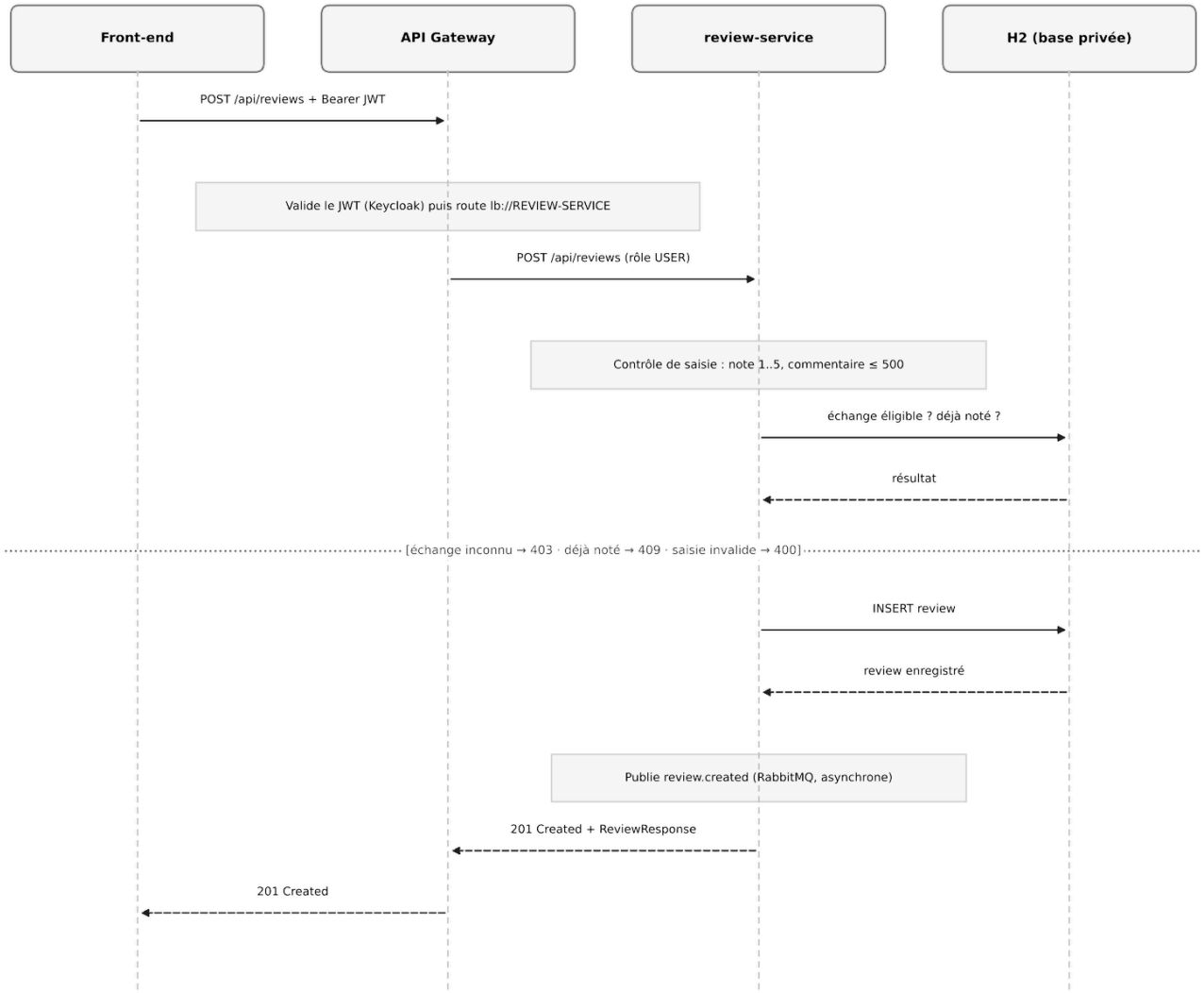


Figure 21 – Création d’un avis (POST /api/reviews)

Exemple de communication asynchrone

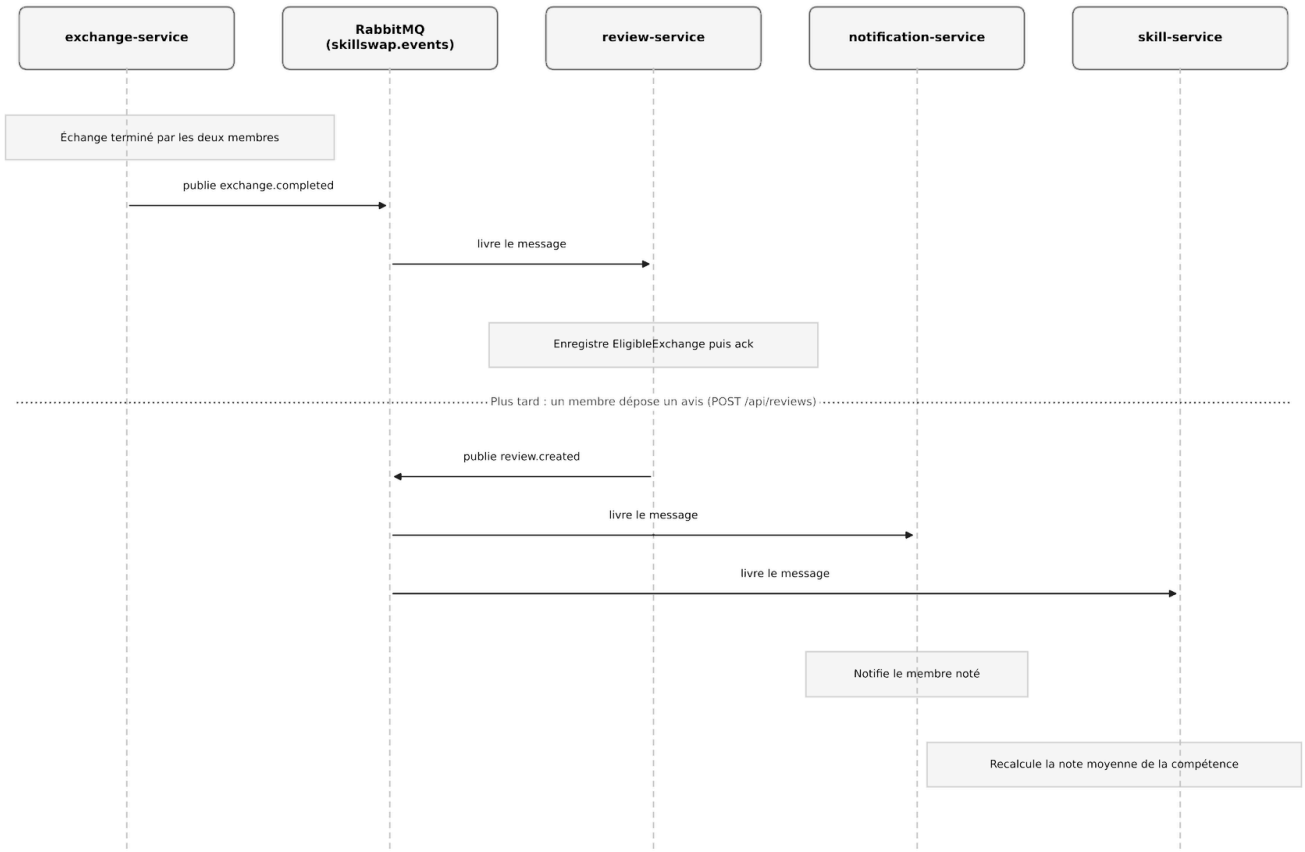


Figure 22 – Éligibilité d’un échange et publication de review.created

7.6 MS6 · notification-service

Description et responsabilités

Rôle. Informe chaque membre de ce qui le concerne sur la plateforme, dans l'application (cloche) et par e-mail. Service entièrement piloté par les événements : aucune notification n'est créée par un appel direct.

- Consommer les événements des autres services (RabbitMQ) ;
- Créer la notification du bon destinataire (dans l'application) ;
- Envoyer l'e-mail correspondant (SMTP) et suivre son statut ;
- Lister mes notifications, filtrer les non lues, compter le badge ;
- Marquer comme lue, tout marquer comme lu ;
- Gérer mes préférences par type (application, e-mail) ;
- Traiter chaque événement une seule fois (idempotence, DLQ).

Tableau 26 – Fiche technique de notification-service

Élément	Valeur
Nom Eureka	NOTIFICATION-SERVICE
Port	8086
Base de données	MySQL notification_db
Messagerie	RabbitMQ (consommateur)
File	notification-service.queue + file morte notification-service.dlq
E-mail	SMTP (MailHog en développement)
Sécurité	JWT Keycloak (USER, ADMIN)
Accès	via la Gateway : /api/notifications/**
Documentation	Swagger

Diagramme de classes – notification-service

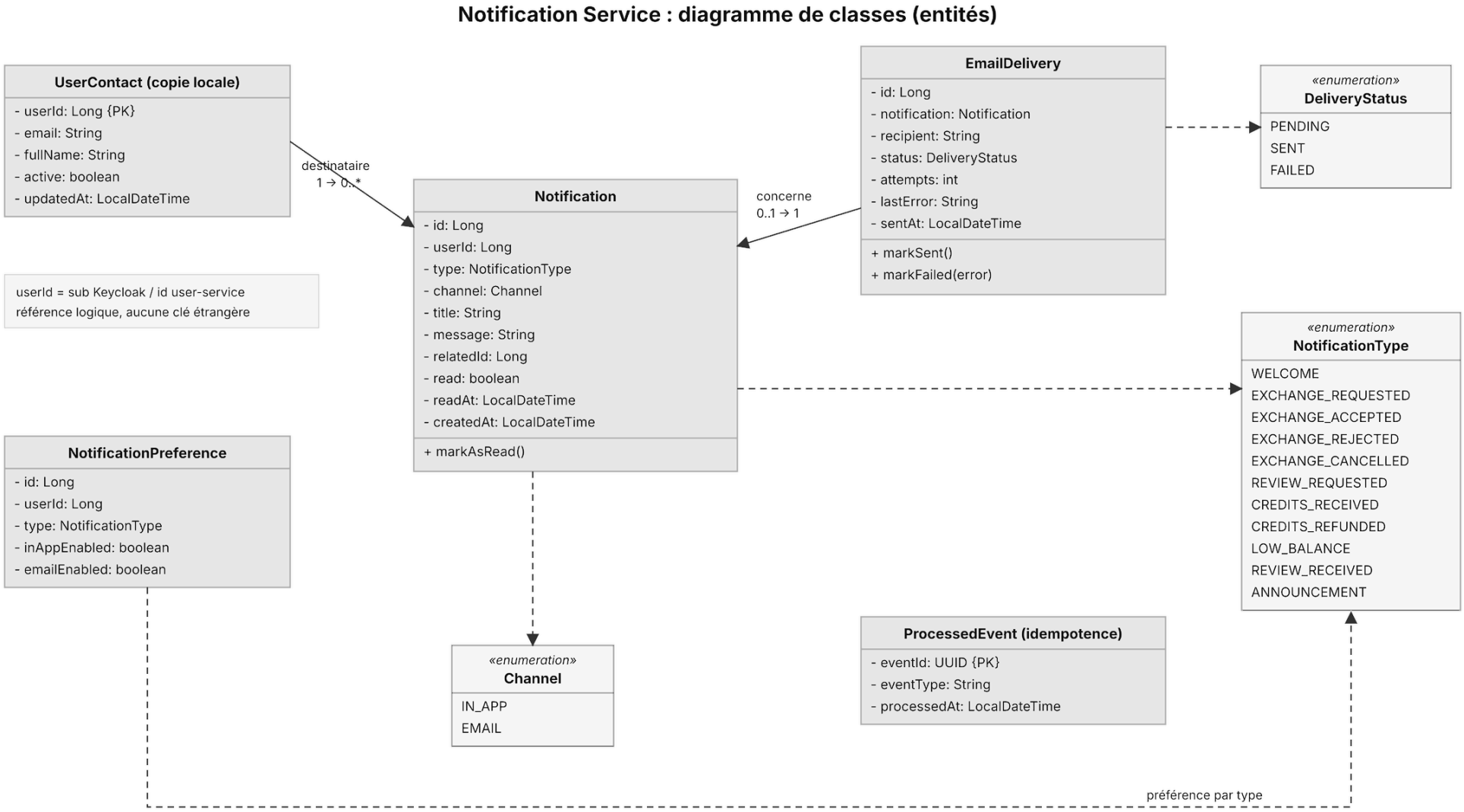


Figure 23 – Diagramme de classes de notification-service

Contrat d'API

Préfixe via l'API Gateway : `/api/notifications` ; authentification par JWT.

Tableau 27 – Principaux endpoints de notification-service

Méthode	Endpoint	Description	Codes HTTP
GET	`/me?unread=&type=&page=&size=`	Lister mes notifications (paginé)	200, 400, 401
GET	`/me/unread-count`	Nombre de non lues (badge)	200, 401
PATCH	`/{id}/read`	Marquer une notification comme lue	200, 401, 403, 404
PATCH	`/me/read-all`	Tout marquer comme lu	204, 401
DELETE	`/{id}`	Supprimer une de mes notifications	204, 401, 403, 404
GET	`/me/preferences`	Consulter mes préférences par type	200, 401
PUT	`/me/preferences`	Modifier mes préférences	200, 400, 401
POST	`/admin/announcements`	Annonce à tous les membres (ADMIN)	202, 400, 401, 403

Codes : 200 OK · 202 Accepted (diffusion asynchrone) · 204 No Content · 400 requête invalide · 401 non authentifié · 403 accès interdit (notification d'un autre membre, rôle ADMIN requis) · 404 introuvable.

Événements RabbitMQ

Tableau 28 – Événements de notification-service

Sens	Événement	Détail
Consommé	`user.registered`	User : « Bienvenue sur SkillSwap » (nouveau membre)
Consommé	`user.deleted`	User : suppression du contact et des notifications
Consommé	`exchange.requested`	Exchange : « Nouvelle demande d'échange » (enseignant)
Consommé	`exchange.accepted`	Exchange : « Votre échange a été accepté » (apprenant)
Consommé	`exchange.rejected`	Exchange : « Votre demande a été refusée » (apprenant)
Consommé	`exchange.cancelled`	Exchange : « Échange annulé » (autre participant)
Consommé	`exchange.completed`	Exchange : « Laissez un avis » (apprenant)
Consommé	`credit.transferred`	Credit : « Vous avez gagné X crédits » (enseignant)
Consommé	`credit.refunded`	Credit : « X crédits vous ont été remboursés » (apprenant)
Consommé	`credit.low-balance`	Credit : « Solde faible » (membre)
Consommé	`review.created`	Review : « Nouvel avis reçu » (membre noté)
Publié	–	Aucun événement publié : consommateur final. Liaisons : user.*, exchange.*, credit.*, review.*. File durable, acquittement manuel, idempotence par eventId (ProcessedEvent), 3 tentatives puis file morte.

Interactions avec les autres microservices

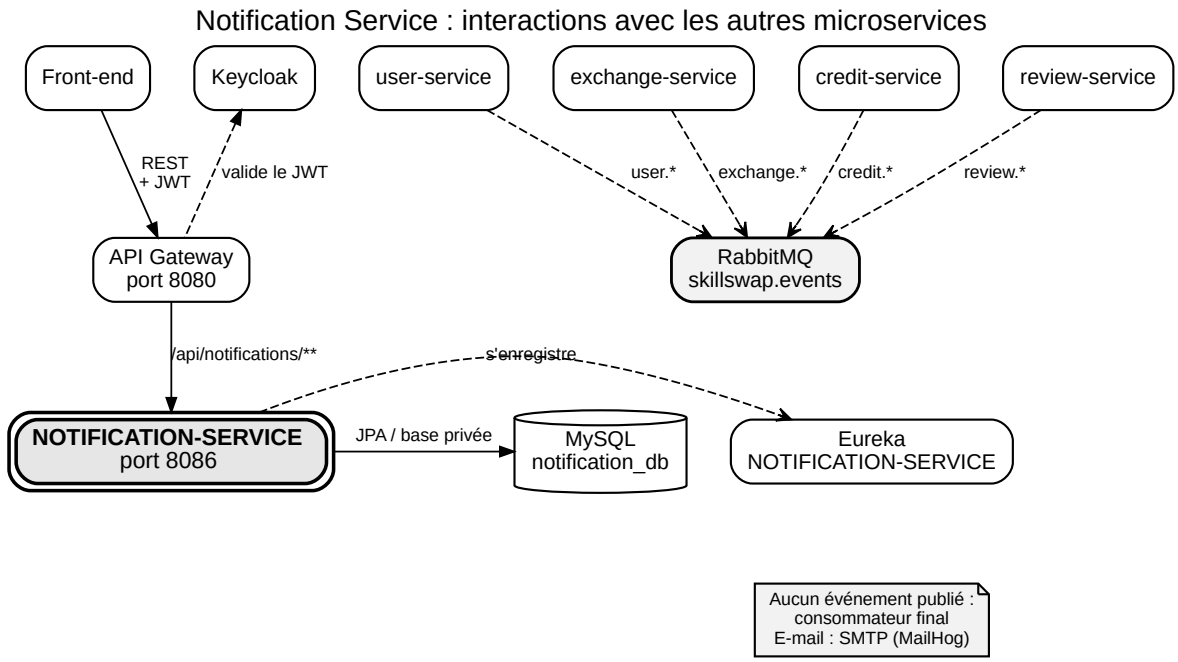


Figure 24 – Interactions de notification-service avec les autres microservices

Exemple de communication synchrone

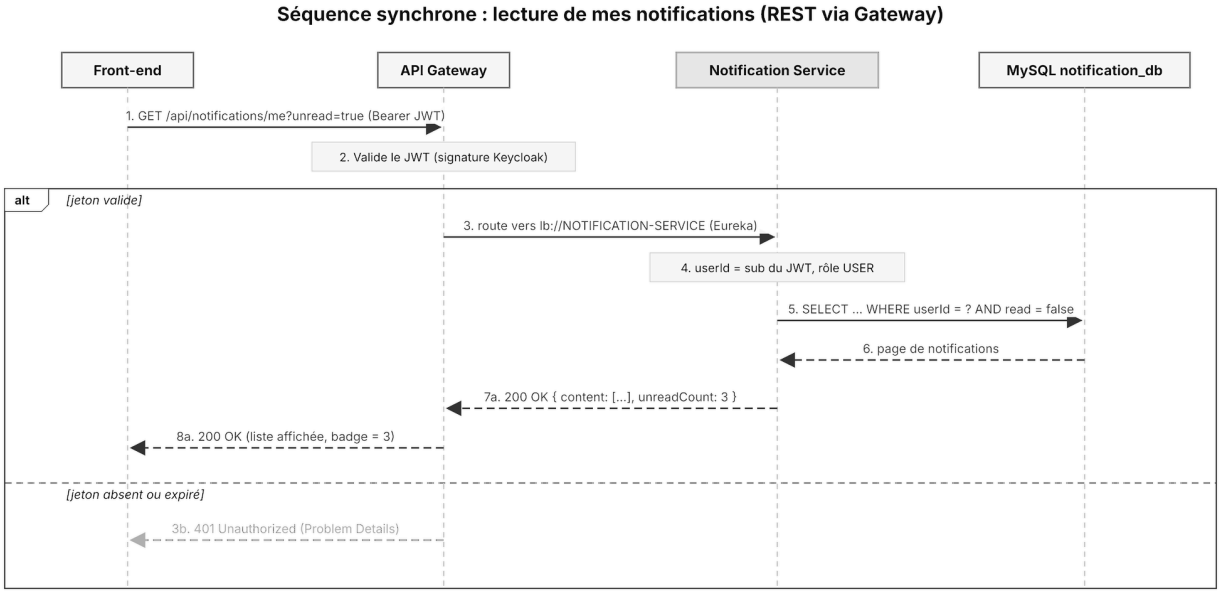


Figure 25 – Marquer une notification comme lue

Exemple de communication asynchrone

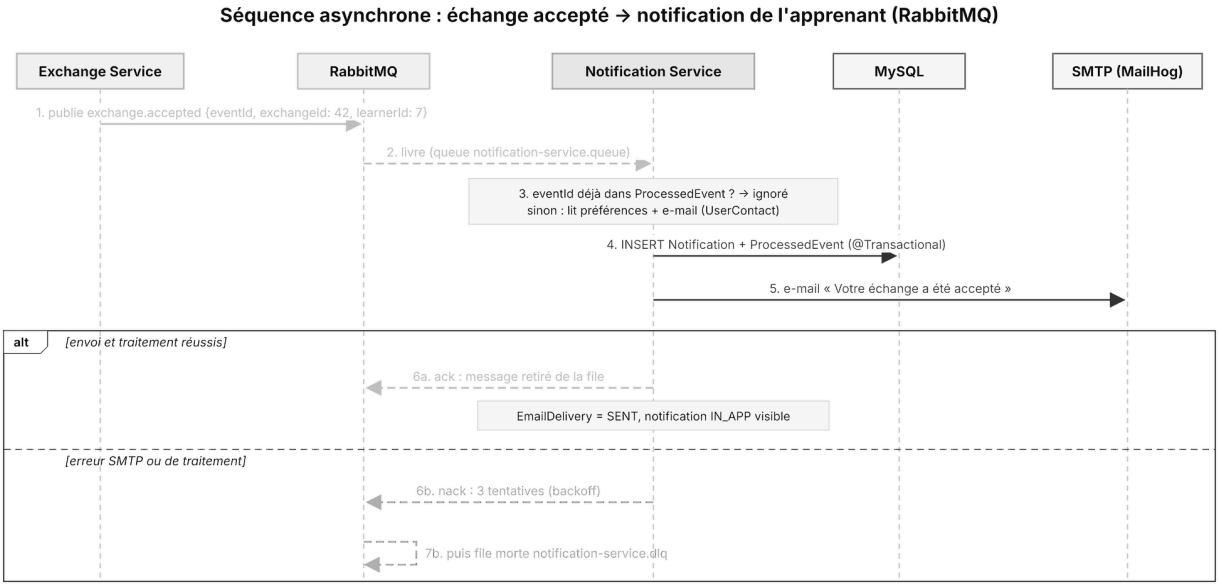


Figure 26 – Traitement d’un événement exchange.requested avec envoi d’e-mail

7.7 MS7 · recommendation-service

Description et responsabilités

Rôle. Maintient un graphe des membres, compétences et échanges, et propose des compétences à apprendre et des enseignants à contacter.

- Le graphe est une **copie dérivée des événements** : il n'est jamais la source de vérité ;
- Compétences à apprendre : celles apprises par les membres qui ont appris les mêmes compétences que moi ;
- Meilleurs enseignants d'une compétence : classés par note moyenne puis nombre d'échanges ;
- Chaque membre ajoute le consommateur d'événements de son domaine.

Tableau 29 – Fiche technique de recommendation-service

Élément	Valeur
Nom Eureka	RECOMMENDATION-SERVICE
Port	8087
Base de données	Neo4j (graphe)
Accès aux données	Spring Data Neo4j, requêtes Cypher
Messagerie	RabbitMQ (consommateur)
Sécurité	JWT Keycloak (USER, ADMIN)
Accès	via la Gateway : /api/recommendations/**
Documentation	Swagger

Diagramme de classes – recommendation-service

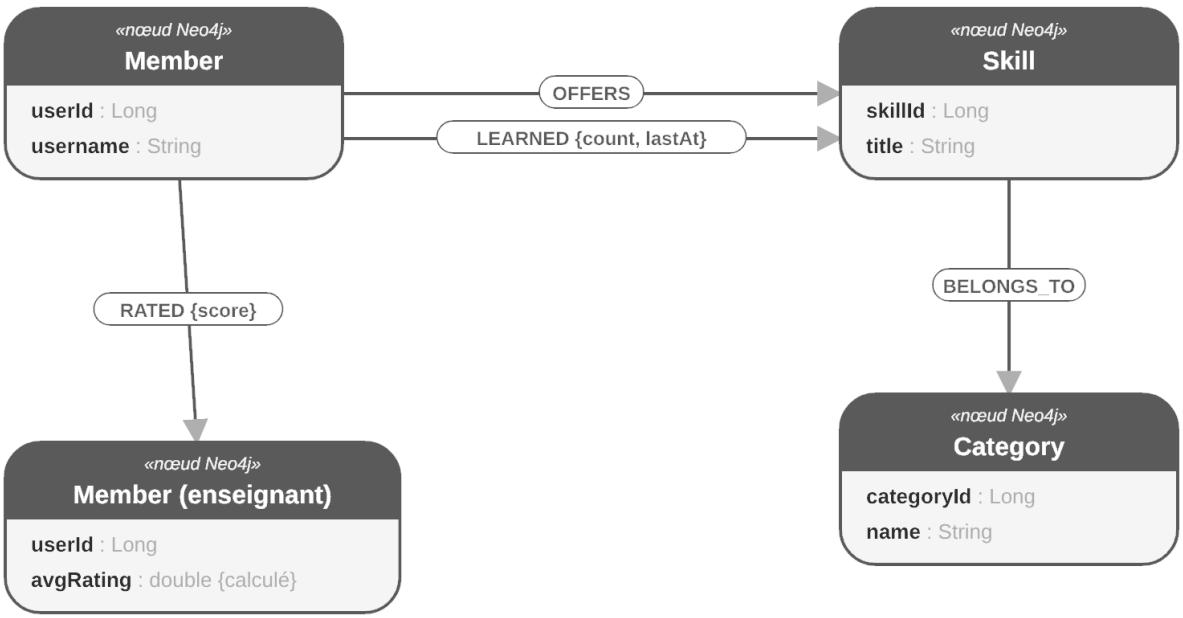


Figure 27 – Modèle de graphe de recommendation-service (diagramme de classes)

Contrat d'API

Préfixe : `/api/recommendations` ; authentification par JWT.

Tableau 30 – Principaux endpoints de recommendation-service

Méthode	Endpoint	Description	Codes HTTP
GET	`/skills?userId=&limit=`	Compétences à apprendre	200, 400, 404
GET	`/teachers?skillId=&limit=`	Meilleurs enseignants d'une compétence	200, 400, 404
GET	`/stats`	Statistiques (ADMIN)	200, 403

Codes : 200 OK · 400 requête invalide · 403 accès interdit · 404 introuvable.

Événements RabbitMQ

Tableau 31 – Événements de recommendation-service

Sens	Événement	Détail
Consommé	`user.registered / deleted`	user-service : crée ou supprime le nœud Member
Consommé	`userskill.created / deleted`	skill-service : crée ou supprime le lien OFFERS
Consommé	`exchange.completed`	exchange-service : crée le lien LEARNED
Consommé	`review.created`	review-service : met à jour le lien RATED
Publié	–	Aucun événement publié.

Interactions avec les autres microservices

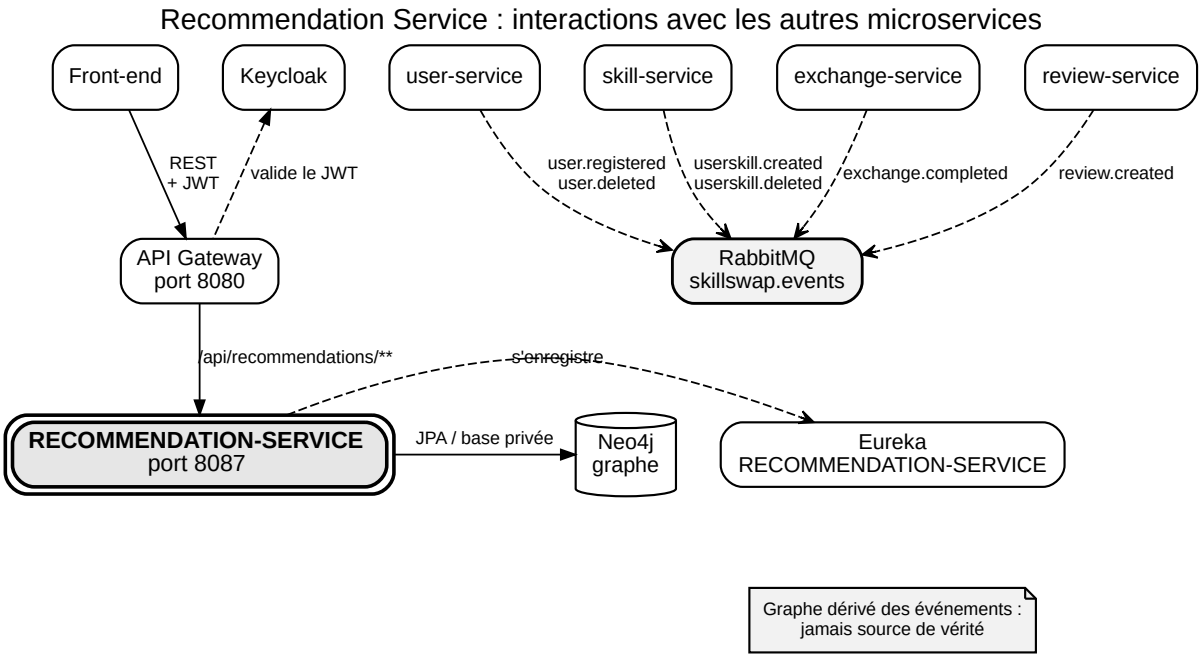


Figure 28 – Interactions de recommendation-service avec les autres microservices

Exemple de communication synchrone

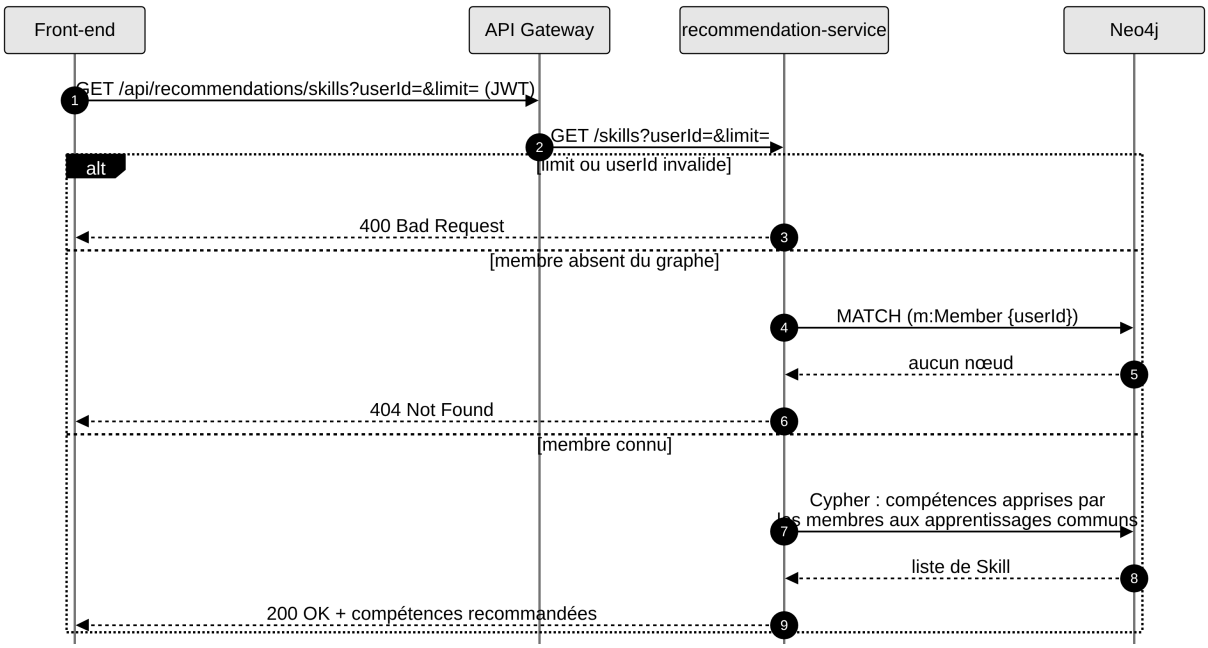


Figure 29 – Recommandation de compétences (GET /skills)

Exemple de communication asynchrone

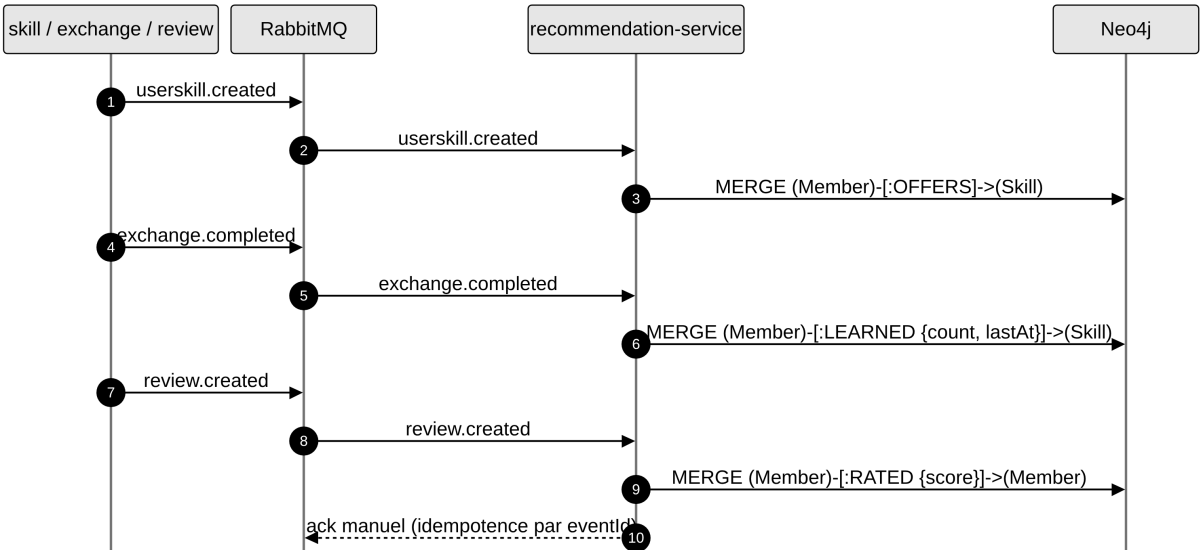


Figure 30 – Mise à jour du graphe à partir des événements

8 Organisation du projet

8.1 Répartition des tâches

Tableau 32 – Répartition des microservices et des tâches transversales

Membre	Microservice	Tâche transversale
Emna Trabelsi	user-service	API Gateway (routes, CORS)
Amal Jemli	skill-service + coordination MS7	Eureka et Config Server
Feryel Bassem	exchange-service	Coordination du front-end Angular
Molka Khlaifi	credit-service	RabbitMQ (exchange, files, DLQ)
Malek Massoudi	review-service	Keycloak (realm, rôles, client)
Wiem Jlassi	notification-service	Docker Compose, README, présentation
Toute l'équipe	recommendation-service (commun)	Chacun ajoute le consommateur de son domaine

8.2 Dépôt Git et gestion de projet

- **Dépôt public** : https://github.com/faryoula11/skillswap-4das6-g5 ; il contient déjà la structure demandée : `Backend/`, `Frontend/`, `Documentation/` et `README.md` ;
- **Branches** : `main` (protégée), `develop`, `feature/<service>` ;
- **Pull Request** relue avant fusion vers `develop` ;
- **GitHub Projects** : tableau Kanban pour le suivi des tâches.

Structure du dépôt (conforme au rendu demandé) :

```
skillswap-4das6-g5/
├── Backend/                          # un projet Maven par microservice
│   ├── user-service/   skill-service/   exchange-service/
│   ├── credit-service/ review-service/  notification-service/
│   └── recommendation-service/  (+ api-gateway, eureka, config-server)
├── Frontend/                         # application Angular
├── Documentation/
│   ├── Presentation/                 # présentation de la séance 4
│   └── Diagrams/                     # architecture globale, classes, séquences
└── README.md                         # sujet, membres, architecture, lancement
```

8.3 Planning prévisionnel

Tableau 33 – Planning prévisionnel (semaines S4 à S10)

Tâche	S4	S5	S6	S7	S8	S9	S10
Conception, Git, présentation	■						
Spring Boot, CRUD, Swagger		■					
Eureka et Config Server		■					
Appels OpenFeign			■				
Modèle Neo4j			■	■			
Événements RabbitMQ				■			
Sécurité Keycloak (JWT)					■		
Front-end Angular					■	■	
Docker Compose complet			■	■	■	■	■
Tests d’intégration						■	
Finalisation et soutenance							■

8.4 Risques et parades

Tableau 34 – Risques identifiés et parades

Risque	Parade
Intégration tardive des services	Contrats OpenAPI dès S4, Docker Compose dès S6, tests chaque semaine
Un service en retard bloque les autres	Contrat d’abord, stubs, appels protégés par un délai de 2 s
Mémoire insuffisante pour tous les conteneurs	Chacun lance son service et l’infrastructure minimale
Neo4j : nouvelle technologie	Modèle minimal, requêtes Cypher simples, démarrage en S6
Incohérence des données entre services	Escrow et remboursement, consommateurs idempotents, file morte

9 Conclusion

Ce dossier présente la conception de SkillSwap : une plateforme d'échange de compétences par crédits-temps reposant sur sept microservices indépendants, une API Gateway, Keycloak pour la sécurité, OpenFeign pour les vérifications synchrones et RabbitMQ pour la propagation des événements. Pour chaque service, la conception (diagramme de classes), le contrat d'API et les interactions ont été définis afin de permettre un développement en parallèle et une intégration progressive.

Les prochaines étapes sont l'enregistrement des services dans Eureka et la configuration centralisée, l'implémentation des appels OpenFeign et des événements RabbitMQ, l'activation de la sécurité Keycloak (JWT et rôles), puis l'intégration du front-end Angular et le déploiement complet avec Docker Compose.

Annexe Ports et accès de l’environnement

Tableau 35 – Ports des composants (environnement de développement)

Composant	Port	Accès
API Gateway	8080	Point d’entrée de toutes les requêtes du front
user-service	8081	via la Gateway : /api/users/**
skill-service	8082	via la Gateway : /api/skills/**, /api/categories/**, /api/user-skills/**
exchange-service	8083	via la Gateway : /api/exchanges/**
credit-service	8084	via la Gateway : /api/credit/**
review-service	8085	via la Gateway : /api/reviews/**
notification-service	8086	via la Gateway : /api/notifications/**
recommendation-service	8087	via la Gateway : /api/recommendations/**
Keycloak	8180	Console d’administration, realm skillswap
Eureka	8761	Tableau de bord des services enregistrés
Config Server	8888	Configuration centralisée
RabbitMQ	5672 / 15672	AMQP / console de gestion